

API du journal de chantier

Sprint 4, tâche descriptive CRUD_JOURNAL_CHANTIER (numéro de backlog non fourni).

Source : captures du 07/10/2026 et lecture seule de `features/chantier/adaptateur.ts`, `types.ts`, `validations.ts` et `regles.ts` du frontend.

1. Film mental et objectif

Visualiser un chef de chantier qui retrouve son brouillon, ajoute les quantités réalisées, enregistre, soumet puis retrouve les signatures du CT et du CP.

Cette répétition mentale prépare l'action ; elle ne remplace pas les tests.

Avancer par petites étapes observables évite de forcer une solution en cas de blocage.

2. Architecture et invariants

Le module existant `apps.chantier` et ses routes `/api/v1/rapports/` restent compatibles.

Les nouvelles routes sont sous `/api/v1/chantier/`. Un rapport couvre un projet et une date, avec plusieurs lots. `RapportJournalier` porte l'identité, les auteurs, les dates et le statut ; un complément `JournalChantier` porte la saisie structurée et les signatures intermédiaires. Les rubriques JSON sont validées par des `serializers` imbriqués ; les UUID de lots et d'activités sont contrôlés en base.

Les données résident dans le schéma du tenant. Chaque accès exige la permission du module et le périmètre projet. Seul l'auteur modifie son brouillon.

BROUILLON -> SOUMIS -> VALIDE_CT -> APPROUVE_CP ; un rejet motivé rend la saisie modifiable. Le statut historique APPROUVE représente la signature finale du CP.

Les rapports soumis sont immuables. La fenêtre de saisie va de J-2 à J, en heure Africa/Abidjan. Les mises à jour et signatures utilisent des transactions et des verrous pour éviter les soumissions concurrentes et les doubles signatures.

Le journal est déclaratif : aucune sortie de stock ni paiement automatique.

3. Guide d'implémentation par la pratique

1. Lire les contrats du frontend et tracer un exemple de payload complet.
2. Créer le modèle de complément, sa migration et les `serializers` imbriqués.
3. Implémenter les services transactionnels, puis les `selectors` de lecture.
4. Brancher les vues et documenter les routes OpenAPI.
5. Tester le parcours réel, puis les refus d'accès et les erreurs de saisie.

Routes principales :

- GET/POST /api/v1/chantier/rapports/
- GET /api/v1/chantier/rapports/preparation/?projet=<uuid>&date=AAAA-MM-JJ
- GET/PATCH/DELETE /api/v1/chantier/rapports/<uuid>/
- PATCH /api/v1/chantier/rapports/<uuid>/draft/
- POST /api/v1/chantier/rapports/<uuid>/soumettre/
- POST /api/v1/chantier/rapports/<uuid>/valider/
- POST /api/v1/chantier/rapports/<uuid>/approuver/
- POST /api/v1/chantier/rapports/<uuid>/rejeter/
- GET /api/v1/chantier/journal/?date_debut=&date_fin=

Un brouillon accepte des champs incomplets et des nombres null. La soumission exige les horaires, la météo, une note de 10 caractères et les rubriques applicables. Les effectifs présents ne dépassent pas les prévus, les retards ne dépassent pas les présents. Une absence ou un retard exige une observation. Une journée d'arrêt ne déclare ni travail, ni personnel, ni consommation.

Photos : 5 maximum ; documents : 3 maximum. Les médias sont servis par une réponse authentifiée du tenant et du projet, jamais par une URL publique.

Les fichiers data: base64 sont validés (type réel des images, signature des documents, taille, extension), puis conservés dans la saisie JSON PostgreSQL.

Chaque fichier est limité à 2 Mio et le rapport complet à 16 Mio. Les images restent directement affichables par le frontend, sans perdre l'authentification sur une balise image. Ce choix convient aux volumes bornés du formulaire ; un stockage binaire privé séparé sera préférable si le volume de médias augmente.

Les prix unitaires sont en centimes FCFA. Leur lecture et leur écriture exigent projets.voir_montants ; les réponses masquent les tarifs avec null pour CC/CT.

La production quantitative peut être soumise sans tarif. Le frontend devra rendre ce champ facultatif pour les utilisateurs sans ce droit. Un null renvoyé par la saisie masquée conserve un tarif préexistant en base.

La validation CT exige chantier.valider et l'affectation CT. L'approbation CP exige chantier.rediger (droit du gabarit CP actuel), l'affectation CP ou la responsabilité chef_projet, et la signature CT préalable. Le droit de rédaction seul ne suffit pas. Un utilisateur ne signe jamais son propre rapport.

Un motif de rejet exige 20 caractères. Le journal d'audit conserve les étapes et commentaires antérieurs, y compris après correction et resoumission.

POST /api/v1/chantier/rapports/<uuid>/alertes/ enregistre une notification interne dédoublée par clé. GET /api/v1/chantier/alertes/ fournit la liste paginée adressée à l'utilisateur (CT/CP). Ce canal est interne : aucun email ou push n'est envoyé.

4. Signal d'erreur et pré-mortem

Hypothèse : un UUID valide serait accessible. Réfutation : tester un projet non affecté et un autre tenant. Hypothèse : deux autosauvegardes seraient sans danger. Réfutation : verrouiller le rapport et refuser les modifications après soumission. Hypothèse : un brouillon complet équivaut à une soumission. Réfutation : vérifier les valeurs null et les contraintes au moment de soumettre. Les erreurs sont des informations qui corrigent le modèle mental (Dehaene), et non une mesure de la valeur du développeur. Examiner les données avant d'adopter la première explication (Kahneman).

5. Checklist de tests et validation

- Création, lecture, PATCH partiel et reprise depuis préparation.
- Doublet projet/date, UUID extérieur, fenêtre J-2, brouillon d'un autre auteur.
- Soumission incomplète refusée sans écriture partielle.
- Rapport soumis verrouillé ; validation CT, approbation CP et rejet motivé.
- Limites de médias et téléchargement protégé.
- Compatibilité avec la suite existante `apps/chantier/tests/test_rapports_api.py`.
- `manage.py check` ; `makemigrations --check --dry-run` ; tests `pytest` ciblés.

6. Défi Homo Docens

Sans ouvrir le code, expliquer à un pair pourquoi un PATCH conserve les sections absentes, pourquoi la validation est différente pour un brouillon et une soumission, et pourquoi un verrou SQL protège une signature. Rejouer le parcours en rappel actif (attention, engagement, retour d'erreur et consolidation), puis vérifier les explications avec les tests. Réutiliser les patterns connus de serializers et transactions pour rendre le nouveau domaine plus familier.

Branchement frontend

`JOURNAL_SIMULE` et `SAISIE_SIMULEE` dans `features/chantier/adaptateur.ts` maintiennent actuellement la simulation. Le développeur devra les basculer une fois les migrations appliquées. Aucun fichier frontend n'est modifié par l'agent. Sous ces routes, le contrat de saisie reprend les noms de `ChargeSaisie` : `projet_id`, `date`, `heure_debut`, `heure_fin`, `arret`, `meteo`, `effectifs`, `presence_sous_traitant`, `production`, `lots_travailes`, `activites`, `materiaux`, `livraisons`, `besoins`, `equipements`, `incidents`, `blocage`, `photos`, `pieces_jointes`,

previsions et note_cc. GET détail renvoie aussi saisie pour reprendre le formulaire.

Le CRUD, la préparation, le journal quotidien, les filtres historiques et le circuit de signatures sont couverts. Les synthèses périodiques agrégées, les relances manuelles et la planification automatique de 17h30 sont des extensions distinctes ; leurs routes proposées par le frontend ne sont pas livrées ici.

Le stock proposé dans préparation est vide : le module stocks n'a pas de modèle concret. Les matériaux peuvent être déclarés librement. Le cumul de quantité provient des journaux soumis antérieurs, avec la valeur initiale du planning, et n'altère pas directement les quantités du planning. Les photographies des rapports soumis conservent les libellés et références connus à la soumission. Le tableau des rapports manquants utilise les jours ouvrés (lundi-vendredi) et les dates actuelles des projets actifs ; il ne reconstitue pas un calendrier historique d'activité, de congés ou de jours fériés.

Activation et validation locale

La migration est apps/chantier/migrations/0005_alertejournal_journalchantier_and_more.py.

Elle crée les compléments et alertes et adapte l'unicité des rapports non supprimés.

Sur un environnement avec historique cohérent :

```
.venv/Scripts/python.exe manage.py migrate_schemas
```

```
.venv/Scripts/python.exe -m pytest apps/chantier/tests/ --reuse-db -q
```

Lors de cette tâche, la base locale ccd_digital présente une incohérence préalable :

tiers.0001_initial est enregistrée avant sa dépendance

accounts.0001_squashed_0018_seed_permission_modules_m2m. Son historique n'a pas été modifié. Le schéma neuf a été migré et les tests ont été exécutés dans test_journal_api_validation, issu de la base isolée journal_api_validation.

Ne pas falsifier les enregistrements django_migrations sans vérifier les tables et les opérations réellement appliquées. La correction de cet historique reste nécessaire avant l'activation sur cette base précise.

Exemple minimal de création (le brouillon peut rester incomplet) :

```
POST /api/v1/chantier/rapports/
```

```
{"projet_id": "<uuid-projet>", "date": "2026-10-07"}
```

Réponse 201 : id, statut BROUILLON, saisie normalisée, enregistre_le, commentaire_rejet et alertes_envoyees. PATCH accepte les seules rubriques modifiées et fusionne les champs météo/blocage ; un tableau fourni remplace ce tableau.

Soumettre accepte le payload complet ou {} si le brouillon est déjà complet.

Réponse : id et reference. La validation CT et l'approbation CP acceptent {"commentaire": "...". Le rejet accepte {"motif": "20 caractères minimum"}. DELETE renvoie 204 sans corps. Les erreurs suivent le format erreur.code, erreur.message, erreur.details et erreur.trace_id déjà commun au projet.

Résultats de vérification

42 tests passent : 25 tests de parcours du journal (paramétrisations comprises), 8 validations pures et 9 tests de compatibilité des anciennes routes.

Ruff passe sur les nouveaux fichiers. Aucune migration supplémentaire n'est détectée par makemigrations --check --dry-run. La vérification des modèles Django et la génération du schéma du journal sont contrôlées séparément.

Le schéma global conserve des erreurs préexistantes de déclaration de serializers dans le module accounts ; les routes du journal disposent de leurs annotations. check --deploy avec les paramètres de test signale cinq avertissements de sécurité (HSTS, redirection HTTPS, qualité de la clé locale et cookies sécurisés) ; ces paramètres de test ne constituent pas une configuration de déploiement.

apps/chantier/models/journal.py

```
"""Complément du rapport historique pour le journal couvrant tout un chantier."""
from django.db import models
from apps.core.models import ModeleBase
class JournalChantier(ModeleBase):
    rapport = models.OneToOneField(
        "chantier.RapportJournalier", on_delete=models.RESTRICT, related_name="journal"
    )
    saisie = models.JSONField(default=dict)
    # Valeurs de référence au moment de la soumission, indépendantes du planning futur.
    photographie = models.JSONField(default=dict)
    valide_ct_par = models.ForeignKey(
        "accounts.Utilisateur",
        null=True,
        blank=True,
        on_delete=models.RESTRICT,
        related_name="journaux_valides_ct",
    )
    valide_ct_le = models.DateTimeField(null=True, blank=True)
    commentaire_ct = models.TextField(blank=True)
    relance_le = models.DateTimeField(null=True, blank=True)
    class Meta:
        db_table = "journal_chantier"
class AlerteJournal(ModeleBase):
    """Notification interne persistante, dédoublée par clé de formulaire."""
    journal = models.ForeignKey(JournalChantier, on_delete=models.RESTRICT, related_name="alertes")
    cle = models.CharField(max_length=100)
    type = models.CharField(max_length=30)
    description = models.TextField()
    destinataires = models.ManyToManyField("accounts.Utilisateur", related_name="alertes_journal")
    class Meta:
        db_table = "alerte_journal"
        constraints = [
            models.UniqueConstraint(fields=["journal", "cle"], name="uq_alerte_journal_cle")
        ]
```

apps/chantier/serializers/journal.py

```
"""Contrat exact de ChargeSaisie (frontend), avec brouillons incomplets."""
from collections.abc import Mapping
from drf_spectacular.utils import extend_schema_serializer
from rest_framework import serializers as s

class StrictSerializer(s.Serializer):
    def to_internal_value(self, data):
        if isinstance(data, Mapping):
            inconnus = set(data) - set(self.fields)
            if inconnus:
                raise s.ValidationError(dict.fromkeys(inconnus, "Champ inconnu ou non modifiable. "))
            return super().to_internal_value(data)
    def texte(max_length=1000):
        return s.CharField(required=False, allow_blank=True, default="", max_length=max_length)
    def nombre(max_value=10**11):
        return s.FloatField(
            required=False, allow_null=True, default=None, min_value=0, max_value=max_value
        )
    def entier():
        return s.IntegerField(
            required=False, allow_null=True, default=None, min_value=0, max_value=32767
        )
    METEOS = ["ENSOLEILLE", "NUAGEUX", "PLUVIEUX", "ORAGEUX", "BRUMEUX"]
    class MeteoSerializer(StrictSerializer):
        matin = s.ChoiceField(choices=METEOS, allow_blank=True, required=False, default="")
        apres_midi = s.ChoiceField(choices=METEOS, allow_blank=True, required=False, default="")
        conditions = s.ChoiceField(
            choices=["FAVORABLES", "DIFFICILES", "ARRET"], allow_blank=True, required=False, default=""
        )
        temperature_min = s.FloatField(
            required=False, allow_null=True, default=None, min_value=-80, max_value=80
        )
        temperature_max = s.FloatField(
            required=False, allow_null=True, default=None, min_value=-80, max_value=80
        )
        humidite = nombre(100)
        vent = texte(200)
        prevision = texte()
        def validate(self, attrs):
            mini, maxi = attrs.get("temperature_min"), attrs.get("temperature_max")
            if mini is not None and maxi is not None and mini > maxi:
                raise s.ValidationError(
                    {"temperature_max": "Doit être supérieure ou égale au minimum."}
                )
            return attrs
    class ArretSerializer(StrictSerializer):
        motif = s.ChoiceField(choices=["INTERPERIES", "JOUR_FERIE", "AUTRE"])
        precision = texte()
    class EffectifSerializer(StrictSerializer):
        categorie = texte(200)
        prevus = entier()
        presents = entier()
        retards = entier()
        heures = nombre(24)
        observation = texte()
        def validate(self, attrs):
            prevus, presents, retards = (attrs.get(k) for k in ("prevus", "presents", "retards"))
            if prevus is not None and presents is not None and presents > prevus:
                raise s.ValidationError(
                    {"presents": "L'effectif présent dépasse le personnel affecté."}
                )
            if retards is not None and presents is not None and retards > presents:
                raise s.ValidationError({"retards": "Les retards dépassent l'effectif présent."})
            return attrs
    class PresenceSerializer(StrictSerializer):
        presence = s.ChoiceField(choices=["PRESENT", "PARTIEL", "ABSENT"], allow_blank=True)
        motif = texte()
```

```

        qualite = s.ChoiceField(choices=["CONFORME", "NON_CONFORME"], allow_blank=True)
        observation = texte()
class ProductionSerializer(StrictSerializer):
    intervenant = texte(200)
    activite_id = s.UUIDField()
    quantite_jour = nombre()
    prix_unitaire = s.IntegerField(
        min_value=0, max_value=10**15, required=False, default=None, allow_null=True
    )
class PointLotSerializer(StrictSerializer):
    lot_id = s.UUIDField()
    observation = texte(5000)
@extend_schema_serializer(component_name="JournalActivite")
class ActiviteSerializer(StrictSerializer):
    activite_id = s.UUIDField()
    quantite_jour = nombre()
    localisation = texte(300)
    observation = texte(5000)
class MatériauSerializer(StrictSerializer):
    designation = texte(200)
    quantite_consommee = nombre()
    unite = texte(30)
class LivraisonSerializer(StrictSerializer):
    fournisseur = texte(200)
    designation = texte(200)
    quantite = texte(100)
    bon_livraison = texte(100)
    heure = texte(5)
    conforme = s.ChoiceField(choices=["CONFORME", "PARTIELLE", "NON_CONFORME"], allow_blank=True)
    observation = texte()
class BesoinSerializer(StrictSerializer):
    nature = s.ChoiceField(choices=["RUPTURE", "URGENT"])
    designation = texte(200)
    quantite = texte(100)
    observation = texte()
class EquipementSerializer(StrictSerializer):
    designation = texte(200)
    reference = texte(100)
    propriete = s.ChoiceField(choices=["ENTREPRISE", "LOCATION"], allow_blank=True)
    utilisation = texte(100)
    operateur = texte(200)
    etat = s.ChoiceField(choices=["BON", "ENTRETIEN", "PANNE"], allow_blank=True)
    duree_arret = nombre(24)
    observation = texte()
class IncidentSerializer(StrictSerializer):
    cle = s.CharField(max_length=100)
    nature = s.ChoiceField(
        choices=[
            "INCIDENT",
            "DIFFICULTE_TECHNIQUE",
            "NON_CONFORMITE",
            "RETARD",
            "INTEMPERIE",
            "ARRET_TRAVAUX",
            "INSTRUCTION",
            "EVENEMENT_PARTICULIER",
        ]
    )
    type = s.ChoiceField(
        choices=["QUALITE", "SECURITE", "MATERIEL", "APPROVISIONNEMENT", "ADMINISTRATIF"],
        allow_blank=True,
    )
    heure_debut = texte(5)
    heure_fin = texte(5)
    gravite = s.ChoiceField(choices=["MINEUR", "SIGNIFICATIF", "GRAVE"], allow_blank=True)
    decide_par = s.ChoiceField(choices=["CC", "CT", "CP"], allow_blank=True)
    description = texte(5000)
    action_entreprise = texte(5000)

```

```

@extend_schema_serializer(component_name="JournalBlocage")
class BlocageSerializer(StrictSerializer):
    niveau = s.ChoiceField(
        choices=["AUCUN", "MINEUR", "SIGNIFICATIF", "BLOQUANT"],
        allow_blank=True,
        required=False,
        default="AUCUN",
    )
    nature = s.ChoiceField(
        choices=[
            "APPROVISIONNEMENT",
            "METEO",
            "MAIN_OEUVRE",
            "TECHNIQUE",
            "ADMINISTRATIF",
            "SECURITE",
        ],
        required=False,
        default=None,
        allow_null=True,
    )
    description = texte(5000)
    impact = texte()
class PhotoSerializer(StrictSerializer):
    cle = s.CharField(max_length=100)
    fichier = s.CharField(max_length=8_000_000)
    legende = texte(500)
    horodatage_utc = s.DateTimeField()
    latitude = s.FloatField(allow_null=True, min_value=-90, max_value=90)
    longitude = s.FloatField(allow_null=True, min_value=-180, max_value=180)
class PieceSerializer(StrictSerializer):
    cle = s.CharField(max_length=100)
    fichier = s.CharField(max_length=15_000_000)
    nom = s.CharField(max_length=200)
    type_mime = s.ChoiceField(
        choices=[
            "application/pdf",
            "application/msword",
            "application/vnd.ms-excel",
            "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
            "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        ]
    )
    taille = s.IntegerField(min_value=1, max_value=2 * 1024 * 1024)
class PrevisionSerializer(StrictSerializer):
    activite = texte(300)
    equipe = texte(200)
    objectif = texte()
    prerequis = texte()
class SaisieJournalSerializer(StrictSerializer):
    projet_id = s.UUIDField()
    date = s.DateField()
    heure_debut = texte(5)
    heure_fin = texte(5)
    arret = ArretSerializer(required=False, allow_null=True, default=None)
    meteo = MeteoSerializer(required=False, default=dict)
    effectifs = EffectifSerializer(many=True, required=False, default=list, max_length=100)
    presence_sous_traitant = PresenceSerializer(required=False, allow_null=True, default=None)
    production = ProductionSerializer(many=True, required=False, default=list, max_length=200)
    lots_travaux = PointLotSerializer(many=True, required=False, default=list, max_length=100)
    activites = ActiviteSerializer(many=True, required=False, default=list, max_length=300)
    materiaux = MatériauSerializer(many=True, required=False, default=list, max_length=200)
    livraisons = LivraisonSerializer(many=True, required=False, default=list, max_length=100)
    besoins = BesoinSerializer(many=True, required=False, default=list, max_length=100)
    equipements = EquipementSerializer(many=True, required=False, default=list, max_length=100)
    incidents = IncidentSerializer(many=True, required=False, default=list, max_length=100)
    blocage = BlocageSerializer(required=False, default=dict)
    photos = PhotoSerializer(many=True, required=False, default=list, max_length=5)

```

```

pieces_jointes = PieceSerializer(many=True, required=False, default=list, max_length=3)
previsions = PrevisionSerializer(many=True, required=False, default=list, max_length=100)
note_cc = texte(10000)
class FiltresJournalSerializer(StrictSerializer):
    projet = s.UUIDField(required=False)
    date = s.DateField(required=False)
    date_debut = s.DateField(required=False)
    date_fin = s.DateField(required=False)
    auteur = s.CharField(required=False)
    statut = s.ChoiceField(
        choices=["BROUILLON", "SOU MIS", "VALIDE_CT", "APPROUVE_CP", "REJETE"], required=False
    )
    def validate(self, attrs):
        if (
            attrs.get("date_debut")
            and attrs.get("date_fin")
            and attrs["date_debut"] > attrs["date_fin"]
        ):
            raise s.ValidationError({"date_fin": "La fin précède le début."})
        if attrs.get("auteur") not in (None, "moi"):
            raise s.ValidationError({"auteur": "Seule la valeur moi est acceptée."})
        return attrs
class SignatureJournalSerializer(StrictSerializer):
    commentaire = texte(5000)
class RejetJournalSerializer(StrictSerializer):
    motif = s.CharField(min_length=20, max_length=5000)
class AlerteJournalSerializer(StrictSerializer):
    cle = s.CharField(max_length=100)
    type = s.ChoiceField(choices=["BLOCAGE_BLOQUANT", "INCIDENT_GRAVE"])
    description = s.CharField(min_length=1, max_length=5000)

```

apps/chantier/services/journal.py

```
"""Écritures atomiques du journal : brouillon, soumission et signatures."""
import base64
import binascii
import io
import json
from collections.abc import Mapping
from datetime import time, timedelta
from zoneinfo import ZoneInfo
from django.db import IntegrityError, transaction
from django.db.models import Q
from django.shortcuts import get_object_or_404
from django.utils import timezone
from PIL import Image
from rest_framework.exceptions import PermissionDenied, ValidationError
from apps.chantier.models import AlerteJournal, JournalChantier, RapportJournalier
from apps.chantier.serializers.journal import SaisieJournalSerializer
from apps.core.enums import ActionAudit, ModeExecution, StatutRapport
from apps.core.exceptions import ErreurMetier, RapportDejaExistant, RapportNonModifiable
from apps.projets.models import Activite, AffectationProjet, Lot, Projet
from apps.projets.services.machine_etats import verifier_statut_projet_pour_ecriture
from apps.projets.services.sante_declencheur import declencher_recalcul_sante
def aujourd'hui():
    return timezone.localtime(timezone=ZoneInfo("Africa/Abidjan"))
def verifier_jour(jour):
    if not aujourd'hui() - timedelta(days=2) <= jour <= aujourd'hui():
        raise ErreurMetier(
            "Le rapport doit dater de J-2, J-1 ou J.", details={"date": "hors_fenetre_saisie"}
        )
def affectations_actives(projet_id):
    return (
        AffectationProjet.objects.filter(
            projet_id=projet_id,
            est_actif=True,
            date_debut__lte=aujourd'hui(),
            utilisateur__supprime_le__isnull=True,
            utilisateur__statut="ACTIF",
        )
        .filter(Q(date_fin__isnull=True) | Q(date_fin__gte=aujourd'hui()))
        .select_related("utilisateur")
    )
def lots_disponibles(projet):
    return Lot.objects.filter(projet=projet, est_actif=True).exclude(statut="CLOTURE")
def verifier_auteur(rapport, utilisateur):
    if rapport.auteur_id != utilisateur.id:
        raise PermissionDenied("Seul l'auteur peut modifier ou soumettre ce rapport.")
def verifier_signataire(journal, utilisateur, role):
    from apps.core.droits import est_dg
    if journal.rapport.auteur_id == utilisateur.id:
        raise PermissionDenied("L'auteur ne peut pas valider son propre rapport.")
    # Les permissions RBAC sont également vérifiées par la vue. Une signature
    # identifie un intervenant réel du circuit, jamais un simple droit d'édition.
    if utilisateur.is_superuser or est_dg(utilisateur):
        return
    if role == "CP" and journal.rapport.projet.chef_projet_id == utilisateur.id:
        return
    if (
        not affectations_actives(journal.rapport.projet_id)
        .filter(utilisateur=utilisateur, role_projet=role)
        .exists()
    ):
        raise PermissionDenied(f"Cette signature exige l'affectation {role} sur le chantier.")
def statut_journal(journal):
    rapport = journal.rapport
    if rapport.statut == StatutRapport.APPROUVE:
        return "APPROUVE_CP"
    if rapport.statut == StatutRapport.SOUMIS and journal.valide_ct_le:
```

```

        return "VALIDE_CT"
    return rapport.statut
def tracer(journal, utilisateur, action, *, avant=None, evenement="", commentaire=""):
    """Conserve les signatures précédentes lorsqu'un rejet est corrigé."""
    from apps.audit.services import journaliser
    journaliser(
        action=action,
        type_entite="JournalChantier",
        entite_id=journal.rapport_id,
        utilisateur_id=utilisateur.id,
        valeur_avant=avant,
        valeur_apres={
            "statut": statut_journal(journal),
            "evenement": evenement,
            "commentaire": commentaire,
        },
    )
def valider_media(ligne, *, photo=False):
    """Uniquement les fichiers embarqués du formulaire ; aucun téléchargement distant."""
    fichier = ligne["fichier"]
    try:
        entete, contenu = fichier.split(",", 1)
        if not entete.startswith("data:") or not entete.endswith(";base64"):
            raise ValueError
        mime = entete[5:-7]
        octets = base64.b64decode(contenu, validate=True)
    except (ValueError, binascii.Error):
        raise ValidationError(
            {"fichier": "Un fichier data: MIME;base64 valide est requis."}
        ) from None
    if not octets or len(octets) > 2 * 1024 * 1024:
        raise ValidationError({"fichier": "Le fichier doit peser entre 1 octet et 2 Mio."})
    if photo:
        if mime not in ("image/jpeg", "image/png", "image/webp"):
            raise ValidationError({"fichier": "Format photo non accepté."})
        try:
            with Image.open(io.BytesIO(octets)) as image:
                if image.width * image.height > 25_000_000 or Image.MIME.get(image.format) != mime:
                    raise ValueError
                image.verify()
        except (ValueError, OSError, Image.DecompressionBombError):
            raise ValidationError({"fichier": "Image invalide ou trop grande."}) from None
        if bool(ligne["latitude"] is None) != bool(ligne["longitude"] is None):
            raise ValidationError(
                {"longitude": "Latitude et longitude doivent être fournies ensemble."}
            )
    else:
        if mime != ligne["type_mime"] or len(octets) != ligne["taille"]:
            raise ValidationError(
                {"fichier": "Le type MIME ou la taille ne correspond pas au contenu."}
            )
        extensions = {
            "application/pdf": (".pdf",),
            "application/msword": (".doc",),
            "application/vnd.ms-excel": (".xls",),
            "application/vnd.openxmlformats-officedocument.wordprocessingml.document": (".docx",),
            "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet": (".xlsx",),
        }
        if mime not in extensions or not ligne["nom"].lower().endswith(extensions[mime]):
            raise ValidationError({"nom": "L'extension du document ne correspond pas à son type."})
        if mime == "application/pdf" and not octets.startswith(b"%PDF-"):
            raise ValidationError({"fichier": "Contenu PDF invalide."})
        if mime in ("application/msword", "application/vnd.ms-excel") and not octets.startswith(
            bytes.fromhex("D0CF11E0A1B11AE1")
        ):
            raise ValidationError({"fichier": "Document Office invalide."})
        if mime.endswith(".document") or mime.endswith(".sheet"):
            import zipfile

```

```

try:
    with zipfile.ZipFile(io.BytesIO(octets)) as archive:
        attendu = (
            "word/document.xml" if mime.endswith(".document") else "xl/workbook.xml"
        )
        if (
            attendu not in archive.namelist()
            or "[Content_Types].xml" not in archive.namelist()
        ):
            raise ValueError
    except (ValueError, zipfile.BadZipFile):
        raise ValidationError({"fichier": "Document Office Open XML invalide."}) from None
def normaliser_saisie(data, *, projet, jour, utilisateur, existante=None):
    if not isinstance(data, Mapping):
        raise ValidationError("Un objet JSON est requis.")
    # PATCH conserve les sections absentes ; les objets météo/blocage sont fusionnés.
    fusion = {**(existante or {}), **data}
    for cle in ("meteo", "blocage"):
        if isinstance(data.get(cle), dict):
            fusion[cle] = {**(existante or {}).get(cle, {}), **data[cle]}
    fusion.setdefault("projet_id", str(projet.id))
    fusion.setdefault("date", jour.isoformat())
    serializer = SaisieJournalSerializer(data=fusion)
    serializer.is_valid(raise_exception=True)
    saisie = dict(serializer.data)
    from apps.core.droits import peut_voir_montants
    if "production" in data and not peut_voir_montants(utilisateur):
        anciens_prix = {
            (ligne["activite_id"], ligne["intervenant"]): ligne["prix_unitaire"]
            for ligne in (existante or {}).get("production", [])
        }
        for ligne in saisie["production"]:
            if ligne["prix_unitaire"] is not None:
                raise ValidationError(
                    {"production": "Le prix exige la permission projets.voir_montants."}
                )
            # Une valeur masquée renvoyée par le formulaire ne doit pas effacer
            # le tarif déjà autorisé en base.
            ligne["prix_unitaire"] = anciens_prix.get((ligne["activite_id"], ligne["intervenant"]))
    if saisie["projet_id"] != str(projet.id) or saisie["date"] != jour.isoformat():
        raise ValidationError("Le projet et la date du rapport sont immuables.")
    # Les valeurs par défaut d'un serializer imbriqué sont appliquées explicitement.
    for cle, classe in (("meteo", "MeteoSerializer"), ("blocage", "BlocageSerializer")):
        from apps.chantier.serializers import journal as contrats
        nested = getattr(contrats, classe)(data=saisie[cle])
        nested.is_valid(raise_exception=True)
        saisie[cle] = dict(nested.data)
    lots = {str(ligne.id): ligne for ligne in lots_disponibles(projet)}
    activites = {
        str(a.id): a
        for a in Activite.objects.filter(lot_id__in=lots, est_actif=True).select_related("lot")
    }
    vus = set()
    for ligne in saisie["lots_travaillés"]:
        identifiant = ligne["lot_id"]
        if identifiant not in lots or identifiant in vus:
            raise ValidationError({"lots_travaillés": "Lot extérieur, inactif ou dupliqué."})
        if lots[identifiant].mode_execution == ModeExecution.SOUS_TRAITANCE_INFORMELLE:
            raise ValidationError(
                {"lots_travaillés": "Ce lot se suit par la production des intervenants."}
            )
        vus.add(identifiant)
    for section in ("activites", "production"):
        lignes_vues = set()
        for ligne in saisie[section]:
            activite = activites.get(ligne["activite_id"])
            cle = (ligne["activite_id"], ligne.get("intervenant", ""))
            if activite is None or cle in lignes_vues:

```

```

        raise ValidationError({section: "Activité extérieure, inactive ou dupliquée."})
    production = activite.lot.mode_execution == ModeExecution.SOUS_TRAITANCE_INFORMELLE
    if production != (section == "production"):
        raise ValidationError({section: "Le mode de suivi ne correspond pas au lot."})
    if section == "activites" and str(activite.lot_id) not in vus:
        raise ValidationError({section: "L'activité doit appartenir à un lot travaillé."})
    lignes_vues.add(cle)
for section in ("incidents", "photos", "pieces_jointes"):
    cles = [ligne["cle"] for ligne in saisie[section]]
    if len(cles) != len(set(cles)):
        raise ValidationError({section: "Les clés de ligne doivent être uniques."})
    # JSON standard : ni NaN ni Infinity, y compris dans les flottants imbriqués.
try:
    json.dumps(saisie, allow_nan=False)
except ValueError:
    raise ValidationError("Les nombres doivent être finis.") from None
for ligne in saisie["photos"]:
    valider_media(ligne, photo=True)
for ligne in saisie["pieces_jointes"]:
    valider_media(ligne)
if len(json.dumps(saisie).encode()) > 16 * 1024 * 1024:
    raise ValidationError("Le rapport dépasse 16 Mio.")
return saisie
def valider_soumission(saisie, projet):
    erreurs = {}
    def exiger(condition, cle, message):
        if not condition:
            erreurs.setdefault(cle, []).append(message)
    def horaire(valeur):
        try:
            return time.fromisoformat(valeur) if len(valeur) == 5 else None
        except ValueError:
            return None
    debut, fin = horaire(saisie["heure_debut"]), horaire(saisie["heure_fin"])
    exiger(debut is not None, "heure_debut", "Un horaire HH:MM est requis.")
    exiger(
        fin is not None and (debut is None or fin > debut),
        "heure_fin",
        "La fin doit être postérieure au début.",
    )
    exiger(
        all(saisie["meteo"].get(k) for k in ("matin", "apres_midi", "conditions")),
        "meteo",
        "Matin, après-midi et conditions sont requis.",
    )
    if saisie["arret"]:
        exiger(
            saisie["arret"]["motif"] != "AUTRE" or bool(saisie["arret"]["precision"]),
            "arret",
            "Préciser le motif de l'arrêt.",
        )
    for cle in (
        "lots_travailles",
        "activites",
        "production",
        "effectifs",
        "equipements",
        "materiaux",
        "livraisons",
        "besoins",
        "incidents",
        "photos",
        "pieces_jointes",
        "previsions",
    ):
        exiger(not saisie[cle], cle, "Une journée d'arrêt ne déclare pas cette rubrique.")
    exiger(
        saisie["presence_sous_traitant"] is None,

```

```

        "presence_sous_traitant",
        "Sans objet pour une journée d'arrêt.",
    )
    exiger(
        saisie["blocage"]["niveau"] == "AUCUN",
        "blocage",
        "Sans objet pour une journée d'arrêt.",
    )
else:
    exiger(
        len(saisie["note_cc"].strip()) >= 10,
        "note_cc",
        "La note doit comporter au moins 10 caractères.",
    )
modes = set(lots_disponibles(projet).values_list("mode_execution", flat=True))
if ModeExecution.REGIE in modes:
    exiger(
        bool(saisie["effectifs"]),
        "effectifs",
        "Au moins une catégorie est requise en régie.",
    )
for ligne in saisie["effectifs"]:
    exiger(
        bool(ligne["categorie"])
        and all(ligne[k] is not None for k in ("prevus", "resents", "heures")),
        "effectifs",
        "Catégorie, prévus, présents et heures sont requis.",
    )
    absence = (
        ligne["prevus"] is not None
        and ligne["resents"] is not None
        and ligne["resents"] < ligne["prevus"]
    )
    exiger(
        not (absence or (ligne["retards"] or 0)) or bool(ligne["observation"]),
        "effectifs",
        "Une absence ou un retard exige un motif.",
    )
points = {p["lot_id"]: p for p in saisie["lots_travaillés"]}
from apps.projets.models import Activite
activites = {
    str(a.id): a
    for a in Activite.objects.filter(
        id__in=[ligne["activite_id"] for ligne in saisie["activites"]]
    ).select_related("lot")
}
for ligne in saisie["activites"]:
    exiger(ligne["quantite_jour"] is not None, "activites", "La quantité est requise.")
for identifiant, point in points.items():
    avance = any(
        (ligne["quantite_jour"] or 0) > 0
        and str(activites[ligne["activite_id"]].lot_id) == identifiant
        for ligne in saisie["activites"]
    )
    exiger(
        avance or bool(point["observation"]),
        "lots_travaillés",
        "Une observation est requise si aucune activité du lot n'a avancé.",
    )
if ModeExecution.SOUS_TRAITANCE_INFORMELLE in modes:
    exiger(
        bool(saisie["production"]),
        "production",
        "La production est requise pour la sous-traitance informelle.",
    )
for ligne in saisie["production"]:
    exiger(
        bool(ligne["intervenant"]) and ligne["quantite_jour"] is not None,

```

```

        "production",
        "Intervenant et quantité sont requis ; ",
        "le tarif est réservé aux détenteurs des montants.",
    )
if ModeExecution.SOUS_TRAITANCE_STRUCTUREE in modes:
    presence = saisie["presence_sous_traitant"] or {}
    exiger(
        bool(presence.get("presence")) and bool(presence.get("qualite")),
        "presence_sous_traitant",
        "Présence et qualité sont requises.",
    )
    exiger(
        presence.get("presence") == "PRESENT" or bool(presence.get("motif")),
        "presence_sous_traitant",
        "Motiver une absence ou présence partielle.",
    )
for ligne in saisie["materiaux"]:
    exiger(
        bool(ligne["designation"])
        and bool(ligne["unite"])
        and ligne["quantite_consommee"] is not None,
        "materiaux",
        "Désignation, unité et quantité sont requises.",
    )
for ligne in saisie["livraisons"]:
    exiger(
        all(ligne[k] for k in ("fournisseur", "designation", "quantite", "conformite")),
        "livraisons",
        "Fournisseur, désignation, quantité et conformité sont requis.",
    )
    exiger(
        ligne["conformite"] == "CONFORME" or bool(ligne["observation"]),
        "livraisons",
        "Motiver une livraison non conforme ou partielle.",
    )
for ligne in saisie["equipements"]:
    exiger(
        all(ligne[k] for k in ("designation", "propriete", "utilisation", "etat")),
        "equipements",
        "Désignation, propriété, utilisation et état sont requis.",
    )
    exiger(
        ligne["etat"] == "BON"
        or (ligne["duree_arret"] is not None and bool(ligne["observation"])),
        "equipements",
        "Préciser la durée et le motif d'immobilisation.",
    )
for ligne in saisie["incidents"]:
    exiger(
        len(ligne["description"]) >= 20
        and bool(ligne["gravite"])
        and bool(ligne["decide_par"])
        and bool(ligne["type"]),
        "incidents",
        "Décrire l'événement en 20 caractères, avec catégorie, gravité et décideur.",
    )
    if ligne["heure_debut"] or ligne["heure_fin"]:
        d, f = horaire(ligne["heure_debut"]), horaire(ligne["heure_fin"])
        exiger(
            d is not None and f is not None and f >= d,
            "incidents",
            "Les horaires de l'événement sont incohérents.",
        )
blocage = saisie["blocage"]
exiger(bool(blocage["niveau"]), "blocage", "L'incidence sur le chantier est requise.")
if blocage["niveau"] != "AUCUN":
    exiger(
        bool(blocage["nature"])
    )

```

```

        and len(blocage["description"]) >= 20
        and bool(blocage["impact"]),
        "blocage",
        "Décrire la nature, l'incidence et le blocage (20 caractères).",
    )
    for ligne in saisie["photos"]:
        exiger(bool(ligne["legende"]), "photos", "La légende est requise.")
    for ligne in saisie["besoins"]:
        exiger(bool(ligne["designation"]), "besoins", "La désignation est requise.")
    for ligne in saisie["previsions"]:
        exiger(
            bool(ligne["activite"]) and bool(ligne["objectif"]),
            "previsions",
            "L'activité et l'objectif sont requis.",
        )
    if erreurs:
        raise ErreurMetier("Le rapport est incomplet pour être soumis.", details=erreurs)
@transaction.atomic
def creer_journal(*, projet, utilisateur, jour, data):
    verifier_jour(jour)
    projet = get_object_or_404(Projet.objects.select_for_update(), pk=projet.pk)
    verifier_statut_projet_pour_ecriture(projet, action="NOUVEAU_RAPPORT")
    saisie = normaliser_saisie(data, projet=projet, jour=jour, utilisateur=utilisateur)
    if RapportJournalier.objects.filter(
        projet=projet, date_rapport=jour, lot__isnull=True
    ).exists():
        raise RapportDejaExistant("Un rapport existe déjà pour ce chantier à cette date.") from None
    try:
        with transaction.atomic():
            rapport = RapportJournalier.objects.create(
                projet=projet,
                date_rapport=jour,
                auteur=utilisateur,
                cree_par=utilisateur,
                statut=StatutRapport.BROUILLON,
                soumis_le=None,
                effectif_present=sum(ligne["presents"] or 0 for ligne in saisie["effectifs"]),
                observations=saisie["note_cc"],
                blocages_critiques=int(saisie["blocage"]["niveau"] == "BLOQUANT"),
            )
            journal = JournalChantier.objects.create(
                rapport=rapport, saisie=saisie, cree_par=utilisateur
            )
    except IntegrityError:
        raise RapportDejaExistant("Un rapport existe déjà pour ce chantier à cette date.") from None
    tracer(journal, utilisateur, ActionAudit.CREATION, evenement="BROUILLON_CREE")
    return journal
def verrouiller(journal):
    # Ordre unique : rapport puis complément, également pour les vues historiques.
    rapport = get_object_or_404(
        RapportJournalier.objects.select_for_update(), pk=journal.rapport_id
    )
    journal = get_object_or_404(JournalChantier.objects.select_for_update(), pk=journal.pk)
    journal.rapport = rapport
    return journal
def enregistrer(journal, utilisateur, data):
    rapport = journal.rapport
    verifier_auteur(rapport, utilisateur)
    if rapport.statut not in (StatutRapport.BROUILLON, StatutRapport.REJETE):
        raise RapportNonModifiable("Un rapport soumis ne peut plus être modifié.")
    verifier_jour(rapport.date_rapport)
    verifier_statut_projet_pour_ecriture(rapport.projet, action="NOUVEAU_RAPPORT")
    ancienne_saisie = journal.saisie
    journal.saisie = normaliser_saisie(
        data,
        projet=rapport.projet,
        jour=rapport.date_rapport,
        utilisateur=utilisateur,
    )

```

```

        existante=journal.saisie,
    )
    journal.save(update_fields=["saisie", "modifie_le"])
    # La saisie est la source de vérité, y compris pour les anciennes listes.
    rapport.effectif_present = sum(ligne["resents"] or 0 for ligne in journal.saisie["effectifs"])
    rapport.observations = journal.saisie["note_cc"]
    rapport.blocages_critiques = int(journal.saisie["blocage"]["niveau"] == "BLOQUANT")
    rapport.save(
        update_fields=["effectif_present", "observations", "blocages_critiques", "modifie_le"]
    )
    if ancienne_saisie != journal.saisie:
        tracer(
            journal,
            utilisateur,
            ActionAudit.MODIFICATION,
            avant={
                "champs_modifies": [
                    cle for cle in journal.saisie if ancienne_saisie.get(cle) != journal.saisie[cle]
                ]
            },
            evenement="BROUILLON_ENREGISTRE",
        )
    return journal
@transaction.atomic
def modifier_journal(*, journal, utilisateur, data):
    return enregistrer(verrouiller(journal), utilisateur, data)
@transaction.atomic
def soumettre_journal(*, journal, utilisateur, data):
    from apps.chantier.selectors.journal import contexte_projet
    journal = verrouiller(journal)
    avant = {"statut": statut_journal(journal)}
    # Stabiliser les modes d'exécution pour la validation et la photographie.
    list(lots_disponibles(journal.rapport.projet).select_for_update().order_by("id"))
    journal = enregistrer(journal, utilisateur, data)
    rapport = journal.rapport
    valider_soumission(journal.saisie, rapport.projet)
    journal.photographie = contexte_projet(rapport.projet, rapport.date_rapport)
    journal.valide_ct_par = None
    journal.valide_ct_le = None
    journal.commentaire_ct = ""
    journal.save()
    rapport.statut = StatutRapport.SOUMIS
    rapport.soumis_le = timezone.now()
    rapport.valide_par = None
    rapport.valide_le = None
    rapport.commentaire_validation = ""
    rapport.save()
    Lot.objects.filter(id__in=[lot["id"] for lot in journal.photographie["lots"]]).update(
        premier_rapport_soumis=True
    )
    declencher_recalcul_sante(
        projet_id=rapport.projet_id,
        declencheur_type="RAPPORT_SOUMISSION",
        declencheur_id=rapport.id,
    )
    tracer(journal, utilisateur, ActionAudit.SIGNATURE, avant=avant, evenement="SOUMISSION_CC")
    return journal
@transaction.atomic
def signer_journal(*, journal, utilisateur, etape, commentaire=""):
    journal = verrouiller(journal)
    verifier_signataire(journal, utilisateur, "CT" if etape == "CT" else "CP")
    verifier_statut_projet_pour_ecriture(journal.rapport.projet)
    attendu = "SOUMIS" if etape == "CT" else "VALIDE_CT"
    if statut_journal(journal) != attendu:
        raise ErreurMetier(f"La signature exige le statut {attendu}.")
    if etape == "CT":
        journal.valide_ct_par = utilisateur
        journal.valide_ct_le = timezone.now()

```

```

        journal.commentaire_ct = commentaire
        journal.save()
    else:
        rapport = journal.rapport
        rapport.statut = StatutRapport.APPROUVE
        rapport.valide_par = utilisateur
        rapport.valide_le = timezone.now()
        rapport.commentaire_validation = commentaire
        rapport.save()
    tracer(
        journal,
        utilisateur,
        ActionAudit.SIGNATURE,
        avant={"statut": attendu},
        evenement=f"SIGNATURE_{etape}",
        commentaire=commentaire,
    )
    return journal
@transaction.atomic
def rejeter_journal(*, journal, utilisateur, motif):
    journal = verrouiller(journal)
    statut = statut_journal(journal)
    if statut not in ("SOUMIS", "VALIDE_CT"):
        raise ErreurMetier("Seul un rapport soumis ou validé par le CT peut être rejeté.")
    verifier_signataire(journal, utilisateur, "CP" if statut == "VALIDE_CT" else "CT")
    verifier_statut_projet_pour_ecriture(journal.rapport.projet)
    if len(motif.strip()) < 20:
        raise ErreurMetier("Le motif de rejet doit comporter au moins 20 caractères.")
    rapport = journal.rapport
    rapport.statut = StatutRapport.REJETE
    rapport.commentaire_validation = motif.strip()
    rapport.valide_par = utilisateur
    rapport.valide_le = timezone.now()
    rapport.save()
    tracer(
        journal,
        utilisateur,
        ActionAudit.VALIDATION,
        avant={"statut": statut},
        evenement="REJET",
        commentaire=motif.strip(),
    )
    return journal
@transaction.atomic
def supprimer_brouillon(*, journal, utilisateur):
    journal = verrouiller(journal)
    verifier_auteur(journal.rapport, utilisateur)
    verifier_statut_projet_pour_ecriture(journal.rapport.projet)
    if journal.rapport.statut != StatutRapport.BROUILLON:
        raise ErreurMetier("Seuls les brouillons peuvent être supprimés.")
    # Exception limitée aux nouveaux brouillons ; la règle historique delete()
    # reste en vigueur pour les rapports archivés et les anciennes routes.
    journal.rapport.supprime_le = timezone.now()
    journal.rapport.supprime_par = utilisateur
    journal.rapport.save(update_fields=["supprime_le", "supprime_par", "modifie_le"])
    journal.delete(utilisateur=utilisateur)
    tracer(journal, utilisateur, ActionAudit.SUPPRESSION, evenement="BROUILLON_SUPPRIME")
@transaction.atomic
def alerter_journal(*, journal, utilisateur, data):
    journal = verrouiller(journal)
    verifier_auteur(journal.rapport, utilisateur)
    verifier_statut_projet_pour_ecriture(journal.rapport.projet)
    if statut_journal(journal) not in ("BROUILLON", "REJETE"):
        raise RapportNonModifiable()
    alerte, creee = AlerteJournal.objects.get_or_create(
        journal=journal, cle=data["cle"], defaults={"creee": utilisateur}
    )
    if not creee and alerte.type != data["type"]:

```

```
        raise ErreurMetier("Cette clé identifie déjà un autre type d'alerte.")
if creee:
    destinataires = [
        a.utilisateur_id
        for a in affectations_actives(journal.rapport.projet_id).filter(
            role_projet__in=["CT", "CP"]
        )
    ]
    if journal.rapport.projet.chef_projet_id:
        destinataires.append(journal.rapport.projet.chef_projet_id)
    alerte.destinataires.set(destinataires)
    tracer(
        journal,
        utilisateur,
        ActionAudit.CREATION,
        evenement=data["type"],
        commentaire=data["description"],
    )
return alerte
```

apps/chantier/selectors/journal.py

```
"""Lectures et projections compatibles avec l'adaptateur du frontend."""
from collections import defaultdict
from datetime import timedelta
from django.db.models import Prefetch, Q
from apps.chantier.models import JournalChantier
from apps.chantier.services.journal import (
    affectations_actives,
    aujourd'hui,
    lots_disponibles,
    statut_journal,
)
from apps.core.enums import ModeExecution
from apps.core.permissions import filtrer_queryset_par_affectations
from apps.projets.models import Activite, Projet
SECTIONS = {
    ModeExecution.REGIE: ["AVANCEMENT", "EFFECTIFS", "MATERIAUX", "LIVRAISONS", "EQUIPEMENTS"],
    ModeExecution.SOUS_TRAITANCE_INFORMELLE: ["PRODUCTION"],
    ModeExecution.SOUS_TRAITANCE_STRUCTUREE: ["AVANCEMENT", "PRESENCE_SOUS_TRAITANT"],
}
def nom(utilisateur):
    return f"{utilisateur.prenom} {utilisateur.nom}".strip() if utilisateur else ""
def journaux_visibles(request):
    qs = (
        JournalChantier.objects.filter(
            rapport__supprime_le__isnull=True,
            rapport__projet__supprime_le__isnull=True,
        )
        .select_related(
            "rapport",
            "rapport__projet",
            "rapport__projet__chef_projet",
            "rapport__auteur",
            "rapport__valide_par",
            "valide_ct_par",
        )
        .prefetch_related("alertes")
    )
    return filtrer_queryset_par_affectations(
        qs, request.user, champ_projet="rapport__projet_id", request=request
    )
def projets_visibles(request):
    return filtrer_queryset_par_affectations(
        Projet.objects.select_related("chef_projet"), request.user, request=request
    )
def pour_auteur(qs, utilisateur):
    return qs.filter(~Q(rapport__statut="BROUILLON") | Q(rapport__auteur=utilisateur))
def filtre_journaux(qs, filtres, utilisateur):
    for cle, champ in (
        ("projet", "rapport__projet_id"),
        ("date", "rapport__date_rapport"),
        ("date_debut", "rapport__date_rapport__gte"),
        ("date_fin", "rapport__date_rapport__lte"),
    ):
        if cle in filtres:
            qs = qs.filter(**{champ: filtres[cle]})
    if filtres.get("auteur") == "moi":
        qs = qs.filter(rapport__auteur=utilisateur)
    statut = filtres.get("statut")
    if statut == "VALIDE_CT":
        qs = qs.filter(rapport__statut="SOUMIS", valide_ct_le__isnull=False)
    elif statut == "SOUMIS":
        qs = qs.filter(rapport__statut="SOUMIS", valide_ct_le__isnull=True)
    elif statut:
        qs = qs.filter(rapport__statut="APPROUVE" if statut == "APPROUVE_CP" else statut)
    return qs.order_by("-rapport__date_rapport", "-modifie_le")
def theorique(debut, fin, jour):
```

```

if not debut or not fin:
    return 0
if jour < debut:
    return 0
if jour >= fin:
    return 100
return round(100 * ((jour - debut).days + 1) / max(1, (fin - debut).days + 1), 2)
def contexte_projet(projet, jour):
    intervenants = dict.fromkeys(("CC", "CT", "CP"), "")
    for affectation in affectations_actives(projet.id).order_by("cree_le"):
        if affectation.role_projet in intervenants and not intervenants[affectation.role_projet]:
            intervenants[affectation.role_projet] = nom(affectation.utilisateur)
    if projet.chef_projet_id:
        intervenants["CP"] = nom(projet.chef_projet)
    lots = list(
        lots_disponibles(projet).prefetch_related(
            Prefetch("activites", queryset=Activite.objects.filter(est_actif=True))
        )
    )
    # Les journaux antérieurs constituent le cumul déclaratif du journal.
    # La quantité du planning n'est pas modifiée par une saisie de journal.
    cumuls = defaultdict(float)
    bases = {}
    precedents = (
        JournalChantier.objects.filter(
            rapport__projet=projet,
            rapport__date_rapport__lt=jour,
            rapport__supprime_le__isnull=True,
            rapport__statut__in=["SOUMIS", "APPROUVE"],
        )
        .order_by("rapport__date_rapport")
        .values_list("saisie", "photographie")
    )
    for saisie, photographie in precedents:
        for activite in photographie.get("activites", []):
            bases.setdefault(activite["activite_id"], activite["cumul_veille"])
        for ligne in saisie.get("activites", []) + saisie.get("production", []):
            cumuls[ligne["activite_id"]] += ligne["quantite_jour"] or 0
    charges_lots, charges_activites, sections = [], [], []
    for lot in lots:
        charges_lots.append(
            {
                "id": str(lot.id),
                "code": lot.code,
                "nom": lot.libelle,
                "mode_execution": "REGIE_DIRECTE"
                if lot.mode_execution == ModeExecution.REGIE
                else lot.mode_execution,
                "projet_id": str(projet.id),
                "projet_nom": projet.nom,
                "projet_reference": projet.reference,
                "chef_chantier_nom": intervenants["CC"],
            }
        )
    for section in SECTIONS.get(lot.mode_execution, []):
        if section not in sections:
            sections.append(section)
    for rang, activite in enumerate(lot.activites.all(), 1):
        identifiant = str(activite.id)
        charges_activites.append(
            {
                "activite_id": identifiant,
                "lot_id": str(lot.id),
                "suivi": "PRODUCTION"
                if lot.mode_execution == ModeExecution.SOUS_TRAITANCE_INFORMELLE
                else "AVANCEMENT",
                "code": f"{lot.code}.{rang:02d}",
                "libelle": activite.libelle,
            }
        )

```

```

        "unite": activite.unite,
        "quantite_prevue": float(activite.quantite_prevue),
        "cumul_veille": bases.get(identifiant, float(activite.quantite_realisee))
        + cumuls[identifiant],
        "avancement_theorique": theorique(
            activite.date_debut_prevue, activite.date_fin_prevue, jour
        ),
    }
    )
return {
    "projet": {
        "id": str(projet.id),
        "nom": projet.nom,
        "reference": projet.reference,
        "chef_chantier_nom": intervenants["CC"],
    },
    "lots": charges_lots,
    "sections": sections,
    "activites": charges_activites,
    # Pas de stock inventé : le module stocks n'a pas encore de modèle concret.
    "materiaux": [],
    "localisation": ", ".join(filter(None, [projet.quartier, projet.ville])),
    "conducteur_travaux_nom": intervenants["CT"],
    "chef_projet_nom": intervenants["CP"],
}
def existant(journal):
    return {
        "id": str(journal.rapport_id),
        "statut": statut_journal(journal),
        "saisie": journal.saisie,
        "enregistre_le": journal.modifie_le.isoformat(),
        "commentaire_rejet": journal.rapport.commentaire_validation
        if journal.rapport.statut == "REJETE"
        else None,
        "alertes_envoyees": [a.cle for a in journal.alertes.all()],
    }
def preparation(projet, jour, *, journal=None, precedent=None):
    contexte = (
        journal.photographie
        if journal
        and journal.photographie
        and statut_journal(journal) not in ("BROUILLON", "REJETE")
        else contexte_projet(projet, jour)
    )
    saisie_precedente = precedent.saisie if precedent else {}
    reprises = {
        "effectifs": [
            {"categorie": ligne["categorie"], "prevus": ligne["prevus"]}
            for ligne in saisie_precedente.get("effectifs", [])
        ],
        "equipements": [
            {k: ligne[k] for k in ("designation", "reference", "propriete", "opérateur")}
            for ligne in saisie_precedente.get("equipements", [])
        ],
        "intervenants": [
            {k: ligne[k] for k in ("intervenante", "activite_id", "prix_unitaire")}
            for ligne in saisie_precedente.get("production", [])
        ],
    }
    return {
        "aujourd'hui": aujourd'hui().isoformat(),
        **{k: contexte[k] for k in ("projet", "lots", "sections", "activites", "materiaux")},
        "reprises": reprises,
        "rapport": existant(journal) if journal else None,
    }
def reference(journal):
    r = journal.rapport
    return f"RJ-{r.projet.reference}-{r.date_rapport:%Y%m%d}"

```

```

def circuit(journal, contexte):
    r = journal.rapport
    statut = statut_journal(journal)
    cc_signe = bool(r.soumis_le)
    ct_signe = bool(journal.valide_ct_le)
    cp_signe = statut == "APPROUVE_CP"
    echeance_ct = (r.soumis_le + timedelta(hours=24)).isoformat() if r.soumis_le else None
    echeance_cp = (
        (journal.valide_ct_le + timedelta(hours=24)).isoformat() if journal.valide_ct_le else None
    )
    rejete = statut == "REJETE"
    return [
        {
            "role": "CC",
            "signataire_nom": nom(r.auteur),
            "etat": "SIGNE" if cc_signe else "EN_ATTENTE",
            "signe_le": r.soumis_le.isoformat() if r.soumis_le else None,
            "echeance": None,
            "commentaire": None,
        },
        {
            "role": "CT",
            "signataire_nom": nom(journal.valide_ct_par) or contexte["conducteur_travaux_nom"],
            "etat": "REJETE"
            if rejete and not ct_signe
            else "SIGNE"
            if ct_signe
            else "EN_ATTENTE"
            if cc_signe
            else "A_VENIR",
            "signe_le": journal.valide_ct_le.isoformat() if ct_signe else None,
            "echeance": echeance_ct,
            "commentaire": r.commentaire_validation
            if rejete and not ct_signe
            else journal.commentaire_ct,
        },
        {
            "role": "CP",
            "signataire_nom": nom(r.valide_par) if cp_signe else contexte["chef_projet_nom"],
            "etat": "REJETE"
            if rejete and ct_signe
            else "SIGNE"
            if cp_signe
            else "EN_ATTENTE"
            if ct_signe
            else "A_VENIR",
            "signe_le": r.valide_le.isoformat() if cp_signe else None,
            "echeance": echeance_cp,
            "commentaire": r.commentaire_validation if cp_signe or (rejete and ct_signe) else None,
        },
    ]

def document(journal, *, contexte=None):
    """Projection ChargeRapport, calculée sans requête par ligne de rubrique."""
    r, saisie = journal.rapport, journal.saisie
    contexte = contexte or journal.photographie or contexte_projet(r.projet, r.date_rapport)
    quantites = defaultdict(float)
    for ligne in saisie["activites"] + saisie["production"]:
        quantites[ligne["activite_id"]] += ligne["quantite_jour"] or 0
    observations = {ligne["activite_id"]: ligne["observation"] for ligne in saisie["activites"]}
    points = {ligne["lot_id"]: ligne["observation"] for ligne in saisie["lots_travaux"]}
    lots = [
        ligne
        for ligne in contexte["lots"]
        if ligne["id"] in points
        or any(
            a["lot_id"] == ligne["id"] and a["activite_id"] in quantites
            for a in contexte["activites"]
        )
    ]

```

```

]
travaux = []
activites = {a["activite_id"]: a for a in contexte["activites"]}
for lot in lots:
    toutes = [a for a in contexte["activites"] if a["lot_id"] == lot["id"]]
    lignes = [
        {
            "libelle": a["libelle"],
            "unite": a["unite"],
            "quantite_prevue": a["quantite_prevue"],
            "cumul_veille": a["cumul_veille"],
            "quantite_jour": quantites[a["activite_id"]],
            "avancement_theorique": a["avancement_theorique"],
            "observation": observations.get(a["activite_id"]),
        }
        for a in toutes
        if a["activite_id"] in quantites
    ]
    avance = sum(
        min(100, 100 * (a["cumul_veille"] + quantites[a["activite_id"]]) / a["quantite_prevue"])
        for a in toutes
        if a["quantite_prevue"] > 0
    )
    travaux.append(
        {
            "lot": lot,
            "avancement": round(avance / len(toutes), 2) if toutes else 0,
            "avancement_theorique": round(
                sum(a["avancement_theorique"] for a in toutes) / len(toutes), 2
            )
            if toutes
            else 0,
            "observation": points.get(lot["id"]),
            "activites": lignes,
        }
    )
blocage = saisie["blocage"]
effectifs = (
    saisie["effectifs"] if "EFFECTIFS" in contexte["sections"] and not saisie["arret"] else None
)
production = (
    [
        {
            "intervenant": ligne["intervenant"],
            "activite": activites[ligne["activite_id"]]["libelle"],
            "unite": activites[ligne["activite_id"]]["unite"],
            "prix_unitaire": ligne["prix_unitaire"],
            "quantite_jour": ligne["quantite_jour"],
            "cumul": activites[ligne["activite_id"]]["cumul_veille"]
            + (ligne["quantite_jour"] or 0),
        }
        for ligne in saisie["production"]
        if ligne["activite_id"] in activites
    ]
    if "PRODUCTION" in contexte["sections"]
    else None
)
return {
    "id": str(r.id),
    "reference": reference(journal) if r.soumis_le else None,
    "date": r.date_rapport.isoformat(),
    "chantier": {
        "projet_id": str(r.projet_id),
        "projet_nom": contexte["projet"]["nom"],
        "projet_reference": contexte["projet"]["reference"],
        "chef_chantier_nom": nom(r.auteur),
    },
    "lots": lots,

```

```

"statut": statut_journal(journal),
"effectif_present": sum(ligne["presents"] or 0 for ligne in effectifs)
if effectifs is not None
else None,
"effectif_prevu": sum(ligne["prevus"] or 0 for ligne in effectifs)
if effectifs is not None
else None,
"avancement": round(sum(ligne["avancement"] for ligne in travaux) / len(travaux), 2)
if travaux
else None,
"avancement_theorique": round(
    sum(ligne["avancement_theorique"] for ligne in travaux) / len(travaux), 2
)
if travaux
else None,
"nb_incidents": len(saisie["incidents"]),
"nb_blocages": int(blocage["niveau"] != "AUCUN"),
"nb_photos": len(saisie["photos"]),
"soumis_le": r.soumis_le.isoformat() if r.soumis_le else None,
"dernier_rapport_le": None,
"relance_le": journal.relance_le.isoformat() if journal.relance_le else None,
"note_chef_chantier": saisie["note_cc"],
"circuit": circuit(journal, contexte),
"localisation": contexte["localisation"],
"conducteur_travaux_nom": contexte["conducteur_travaux_nom"],
"chef_projet_nom": contexte["chef_projet_nom"],
"heure_debut": saisie["heure_debut"],
"heure_fin": saisie["heure_fin"],
"arret": saisie["arret"],
"meteo": saisie["meteo"],
"effectifs": effectifs,
"production": production,
"travaux": travaux,
"matériaux": [
    {
        "designation": ligne["designation"],
        "unite": ligne["unite"],
        "stock_debut": None,
        "livre": 0,
        "utilise": ligne["quantite_consommee"],
        "seuil_alerte": None,
    }
    for ligne in saisie["matériaux"]
],
"livraisons": saisie["livraisons"],
"besoins": saisie["besoins"],
"equipements": saisie["equipements"],
"incidents": [
    {
        **ligne,
        "numero": f"INC-{i:02d}",
        "action": ligne["action_entreprise"],
        "resolu": False,
    }
    for i, ligne in enumerate(saisie["incidents"], 1)
],
"blocages": [
    {
        **blocage,
        "numero": "BLC-01",
        "escalade": "CP"
        if blocage["niveau"] == "BLOQUANT"
        else "CT"
        if blocage["niveau"] == "SIGNIFICATIF"
        else None,
    }
]
if blocage["niveau"] != "AUCUN"

```

```

else [],
"photos": [
    {
        **ligne,
        "url": ligne["fichier"],
        "heure": ligne["horodatage_utc"][11:16],
        "gps_confirme": ligne["latitude"] is not None and ligne["longitude"] is not None,
    }
    for ligne in saisie["photos"]
],
"pieces_jointes": saisie["pieces_jointes"],
"previsions": saisie["previsions"],
"saisie": saisie,
"enregistre_le": journal.modifie_le.isoformat(),
}
ENTREE_CHAMPS = (
    "id",
    "reference",
    "date",
    "chantier",
    "lots",
    "statut",
    "effectif_present",
    "effectif_prevu",
    "avancement",
    "avancement_theorique",
    "nb_incidents",
    "nb_blocages",
    "nb_photos",
    "soumis_le",
    "dernier_rapport_le",
    "relance_le",
    "note_chef_chantier",
    "circuit",
)
def entree(journal, *, contexte=None):
    charge = document(journal, contexte=contexte)
    return {cle: charge[cle] for cle in ENTREE_CHAMPS}
def entree_manquante(projet, jour, contexte, *, dernier=None):
    return {
        "id": f"{projet.id}_{jour.isoformat()}",
        "reference": None,
        "date": jour.isoformat(),
        "chantier": {
            "projet_id": str(projet.id),
            "projet_nom": projet.nom,
            "projet_reference": projet.reference,
            "chef_chantier_nom": contexte["projet"]["chef_chantier_nom"],
        },
        "lots": contexte["lots"],
        "statut": "NON_SOUMIS",
        "effectif_present": None,
        "effectif_prevu": None,
        "avancement": None,
        "avancement_theorique": None,
        "nb_incidents": None,
        "nb_blocages": None,
        "nb_photos": None,
        "soumis_le": None,
        "dernier_rapport_le": dernier,
        "relance_le": None,
        "note_chef_chantier": None,
        "circuit": [],
    }
def journal_periode(request, debut, fin, *, projet_id=None):
    projets = projets_visibles(request).filter(statut__in=["EN_COURS", "EN_RETARD", "CRITIQUE"])
    if projet_id:
        projets = projets.filter(pk=projet_id)

```

```

        # Inclure aussi les projets achevés avec des rapports dans l'intervalle.
qs = journaux_visibles(request).filter(rapport__date_rapport__range=[debut, fin])
if projet_id:
    qs = qs.filter(rapport__projet_id=projet_id)
rapports = list(qs)
par_jour = {(j.rapport.projet_id, j.rapport.date_rapport): j for j in rapports}
charges = []
for projet in projets:
    contexte = contexte_projet(projet, fin)
    premier = max(
        debut,
        projet.date_debut_reelle
        or projet.date_debut_prevue
        or aujourd'hui() - timedelta(days=2),
    )
    dernier = min(fin, aujourd'hui(), projet.date_fin_reelle or fin)
    for offset in range(max(0, (dernier - premier).days + 1)):
        jour = premier + timedelta(days=offset)
        if jour.weekday() >= 5 and (projet.id, jour) not in par_jour:
            continue
        j = par_jour.pop((projet.id, jour), None)
        if j and (j.rapport.statut != "BROUILLON" or j.rapport.auteur_id == request.user.id):
            # Une photographie d'une soumission a priorité sur le contexte vivant.
            charges.append(entree(j, contexte=j.photographie or contexte))
        else:
            charges.append(entree_manquante(projet, jour, contexte))
for j in par_jour.values():
    if j.rapport.statut != "BROUILLON" or j.rapport.auteur_id == request.user.id:
        charges.append(entree(j))
return sorted(
    charges, key=lambda c: (c["date"], c["chantier"]["projet_reference"]), reverse=True
)

```

apps/chantier/views/journal.py

```
"""API du journal complet, sous /api/v1/chantier/."""
from copy import deepcopy
from datetime import timedelta
from django.shortcuts import get_object_or_404
from drf_spectacular.types import OpenApiTypes
from drf_spectacular.utils import extend_schema
from rest_framework import status
from rest_framework.exceptions import ValidationError
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView
from apps.chantier.models import AlerteJournal
from apps.chantier.permissions import (
    PeutConsulterRapports,
    PeutRedigerRapports,
    PeutValiderRapports,
)
from apps.chantier.selectors.journal import (
    document,
    existant,
    filtre_journaux,
    journal_periode,
    journaux_visibles,
    pour_auteur,
    preparation,
    projets_visibles,
    reference,
)
from apps.chantier.serializers.journal import (
    AlerteJournalSerializer,
    FiltresJournalSerializer,
    RejetJournalSerializer,
    SaisieJournalSerializer,
    SignatureJournalSerializer,
)
from apps.chantier.services.journal import (
    alerter_journal,
    aujourd'hui,
    creer_journal,
    modifier_journal,
    rejeter_journal,
    signer_journal,
    soumettre_journal,
    statut_journal,
    supprimer_brouillon,
)
from apps.core.exceptions import ErreurConflit
def filtres(request):
    serializer = FiltresJournalSerializer(data=request.query_params)
    serializer.is_valid(raise_exception=True)
    return serializer.validated_data
def charger_journal(request, pk):
    return get_object_or_404(pour_auteur(journaux_visibles(request), request.user), rapport_id=pk)
def charge_autorisee(request, charge):
    """Masque les tarifs dans le document, la saisie et les reprises (E-11)."""
    from apps.core.droits import peut_voir_montants
    if peut_voir_montants(request.user, request):
        return charge
    charge = deepcopy(charge)
def masquer(valeur):
    if isinstance(valeur, dict):
        for cle, contenu in valeur.items():
            if cle == "prix_unitaire":
                valeur[cle] = None
    else:
        masquer(contenu)
```

```

        elif isinstance(valeur, list):
            for contenu in valeur:
                masquer(contenu)
    masquer(charge)
    return charge

class JournalRapportsView(APIView):
    def get_permissions(self):
        return [
            IsAuthenticated(),
            PeutRedigerRapports() if self.request.method == "POST" else PeutConsulterRapports(),
        ]
    @extend_schema(
        tags=["Journal de chantier"],
        parameters=[FiltresJournalSerializer],
        responses=OpenApiTypes.OBJECT,
        summary="Lister les rapports du journal",
        operation_id="journal_rapports_liste",
    )
    def get(self, request):
        qs = filtre_journaux(
            pour_auteur(journaux_visibles(request), request.user), filtres(request), request.user
        )
        qs = qs.defer("saisie", "photographie").prefetch_related(None)
        return Response(
            {
                "aujourd'hui": aujourd'hui().isoformat(),
                "resultats": [
                    {
                        "id": str(j.rapport_id),
                        "date": j.rapport.date_rapport.isoformat(),
                        "statut": statut_journal(j),
                        "enregistre_le": j.modifie_le.isoformat(),
                    }
                    for j in qs
                ],
            }
        )
    @extend_schema(
        tags=["Journal de chantier"],
        request=SaisieJournalSerializer,
        responses={201: OpenApiTypes.OBJECT},
        summary="Cr  er le brouillon complet d'un chantier",
    )
    def post(self, request):
        # Valider UUID et date avant toute requ  te ORM pour   viter une erreur 500.
        serializer = SaisieJournalSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        projet = get_object_or_404(
            projets_visibles(request), pk=serializer.validated_data["projet_id"]
        )
        self.check_object_permissions(request, projet)
        journal = creer_journal(
            projet=projet,
            utilisateur=request.user,
            jour=serializer.validated_data["date"],
            data=request.data,
        )
        return Response(
            charge_autorisee(request, existant(journal)), status=status.HTTP_201_CREATED
        )

class JournalPreparationView(APIView):
    permission_classes = [IsAuthenticated, PeutConsulterRapports]
    @extend_schema(
        tags=["Journal de chantier"],
        parameters=[FiltresJournalSerializer],
        responses=OpenApiTypes.OBJECT,
        summary="Pr  parer la saisie : lots, activit  s, reprises et brouillon",
    )

```

```

def get(self, request):
    params = filters(request)
    if "projet" not in params or "date" not in params:
        raise ValidationError("projet et date sont requis.")
    projet = get_object_or_404(projets_visibles(request), pk=params["projet"])
    self.check_object_permissions(request, projet)
    qs = journaux_visibles(request).filter(rapport__projet=projet)
    journal = qs.filter(rapport__date_rapport=params["date"]).first()
    if (
        journal
        and journal.rapport.statut == "BROUILLON"
        and journal.rapport.auteur_id != request.user.id
    ):
        raise ErreurConflit(
            "Un autre auteur a déjà commencé le rapport de ce chantier pour ce jour."
        )
    precedent = (
        pour_auteur(qs, request.user)
        .filter(rapport__date_rapport__lt=params["date"])
        .order_by("-rapport__date_rapport")
        .first()
    )
    return Response(
        charge_autorisee(
            request, preparation(projet, params["date"], journal=journal, precedent=precedent)
        )
    )

class JournalRapportDetailView(APIView):
    def get_permissions(self):
        return [
            IsAuthenticated(),
            PeutConsulterRapports()
            if self.request.method in ("GET", "HEAD", "OPTIONS")
            else PeutRedigerRapports(),
        ]
    @extend_schema(
        tags=["Journal de chantier"],
        responses=OpenApiTypes.OBJECT,
        summary="Lire toutes les rubriques du rapport",
        operation_id="journal_rapport_detail",
    )
    def get(self, request, pk):
        journal = charger_journal(request, pk)
        self.check_object_permissions(request, journal.rapport)
        return Response(charge_autorisee(request, document(journal)))
    @extend_schema(
        tags=["Journal de chantier"],
        request=SaisieJournalSerializer,
        responses=OpenApiTypes.OBJECT,
        summary="Modifier partiellement un brouillon ou un rapport rejeté",
    )
    def patch(self, request, pk):
        journal = charger_journal(request, pk)
        self.check_object_permissions(request, journal.rapport)
        journal = modifier_journal(journal=journal, utilisateur=request.user, data=request.data)
        return Response(charge_autorisee(request, existant(journal)))
    @extend_schema(
        tags=["Journal de chantier"],
        responses={204: None},
        summary="Supprimer logiquement un brouillon de l'auteur",
    )
    def delete(self, request, pk):
        journal = charger_journal(request, pk)
        self.check_object_permissions(request, journal.rapport)
        supprimer_brouillon(journal=journal, utilisateur=request.user)
        return Response(status=status.HTTP_204_NO_CONTENT)

class JournalDraftView(APIView):
    permission_classes = [IsAuthenticated, PeutRedigerRapports]

```

```

    @extend_schema(
        tags=["Journal de chantier"],
        request=SaisieJournalSerializer,
        responses=OpenApiTypes.OBJECT,
        summary="Autosauvegarder le brouillon (PATCH)",
    )
    def patch(self, request, pk):
        journal = charger_journal(request, pk)
        self.check_object_permissions(request, journal.rapport)
        journal = modifier_journal(journal=journal, utilisateur=request.user, data=request.data)
        return Response(charge_autorisee(request, existant(journal)))

class JournalSoumettreView(APIView):
    permission_classes = [IsAuthenticated, PeutRedigerRapports]
    @extend_schema(
        tags=["Journal de chantier"],
        request=SaisieJournalSerializer,
        responses=OpenApiTypes.OBJECT,
        summary="Enregistrer et soumettre atomiquement le rapport",
    )
    def post(self, request, pk):
        journal = charger_journal(request, pk)
        self.check_object_permissions(request, journal.rapport)
        journal = soumettre_journal(journal=journal, utilisateur=request.user, data=request.data)
        return Response({"id": str(journal.rapport_id), "reference": reference(journal)})

class JournalValiderView(APIView):
    permission_classes = [IsAuthenticated, PeutValiderRapports]
    etape = "CT"
    @extend_schema(
        tags=["Journal de chantier"],
        request=SignatureJournalSerializer,
        responses=OpenApiTypes.OBJECT,
        summary="Signer le rapport au niveau CT",
    )
    def post(self, request, pk):
        journal = charger_journal(request, pk)
        self.check_object_permissions(request, journal.rapport)
        serializer = SignatureJournalSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        journal = signer_journal(
            journal=journal,
            utilisateur=request.user,
            etape=self.etape,
            commentaire=serializer.validated_data["commentaire"],
        )
        return Response(charge_autorisee(request, document(journal)))

class JournalApprouverView(JournalValiderView):
    etape = "CP"
    # Le gabarit CP dispose de chantier.rediger (niveau 2), pas de
    # chantier.valider (niveau CT). Le service exige en plus le vrai CP du
    # projet et une signature CT existante ; le droit de rédaction seul
    # n'autorise donc jamais l'approbation.
    permission_classes = [IsAuthenticated, PeutRedigerRapports]
    @extend_schema(
        tags=["Journal de chantier"],
        request=SignatureJournalSerializer,
        responses=OpenApiTypes.OBJECT,
        summary="Approuver le rapport au niveau CP après signature CT",
    )
    def post(self, request, pk):
        return super().post(request, pk)

class JournalRejeterView(APIView):
    permission_classes = [IsAuthenticated, PeutValiderRapports | PeutRedigerRapports]
    @extend_schema(
        tags=["Journal de chantier"],
        request=RejetJournalSerializer,
        responses=OpenApiTypes.OBJECT,
        summary="Rejeter au niveau attendu du circuit avec motif",
    )

```

```

def post(self, request, pk):
    journal = charger_journal(request, pk)
    self.check_object_permissions(request, journal.rapport)
    serializer = RejetJournalSerializer(data=request.data)
    serializer.is_valid(raise_exception=True)
    journal = rejeter_journal(
        journal=journal, utilisateur=request.user, motif=serializer.validated_data["motif"]
    )
    return Response(charge_autorisee(request, document(journal)))

class JournalAlertesView(APIView):
    permission_classes = [IsAuthenticated, PeutRedigerRapports]
    @extend_schema(
        tags=["Journal de chantier"],
        request=AlerteJournalSerializer,
        responses=OpenApiTypes.OBJECT,
        summary="Enregistrer une alerte interne dédoublée pour CT et CP",
    )
    def post(self, request, pk):
        journal = charger_journal(request, pk)
        self.check_object_permissions(request, journal.rapport)
        serializer = AlerteJournalSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        alerte = alerter_journal(
            journal=journal, utilisateur=request.user, data=serializer.validated_data
        )
        return Response({"envoyee_le": alerte.cree_le.isoformat(), "canal": "INTERNE"})

class JournalNotificationsView(APIView):
    permission_classes = [IsAuthenticated, PeutConsulterRapports]
    @extend_schema(
        tags=["Journal de chantier"],
        responses=OpenApiTypes.OBJECT,
        summary="Lire les alertes internes adressées à l'utilisateur",
    )
    def get(self, request):
        from apps.core.pagination import PaginationStandard
        qs = (
            AlerteJournal.objects.filter(
                destinataires=request.user,
                journal_id__in=journaux_visibles(request).values("id"),
            )
            .select_related("journal")
            .order_by("-cree_le")
        )
        pagination = PaginationStandard()
        page = pagination.paginate_queryset(qs, request, view=self)
        return pagination.get_paginated_response(
            [
                {
                    "id": str(alerte.id),
                    "rapport_id": str(alerte.journal.rapport_id),
                    "cle": alerte.cle,
                    "type": alerte.type,
                    "description": alerte.description,
                    "envoyee_le": alerte.cree_le.isoformat(),
                }
                for alerte in page
            ]
        )

class JournalView(APIView):
    permission_classes = [IsAuthenticated, PeutConsulterRapports]
    @extend_schema(
        tags=["Journal de chantier"],
        parameters=[FiltresJournalSerializer],
        responses=OpenApiTypes.OBJECT,
        summary="Lire le journal, historique et rapports manquants",
    )
    def get(self, request):
        params = filtres(request)

```

```
debut = params.get("date_debut", aujourd'hui() - timedelta(days=62))
fin = params.get("date_fin", aujourd'hui())
if (fin - debut).days > 366 or fin < debut:
    raise ValidationError("La période doit durer au plus 366 jours, avec fin >= début.")
if params.get("projet"):
    projet = get_object_or_404(projets_visibles(request), pk=params["projet"])
    self.check_object_permissions(request, projet)
from django.utils import timezone
return Response(
    {
        "aujourd'hui": aujourd'hui().isoformat(),
        "lu_le": timezone.now().isoformat(),
        "resultats": journal_periode(request, debut, fin, projet_id=params.get("projet")),
    }
)
```

apps/chantier/urls_journal.py

```
"""Les anciennes URL /rapports/ restent disponibles sans changement de contrat."""
from django.urls import path
from apps.chantier.views.journal import (
    JournalAlertesView,
    JournalApprouverView,
    JournalDraftView,
    JournalNotificationsView,
    JournalPreparationView,
    JournalRapportDetailView,
    JournalRapportsView,
    JournalRejeterView,
    JournalSoumettreView,
    JournalValiderView,
    JournalView,
)
app_name = "journal_chantier"
urlpatterns = [
    path("alertes/", JournalNotificationsView.as_view(), name="notifications"),
    path("journal/", JournalView.as_view(), name="journal"),
    path("rapports/", JournalRapportsView.as_view(), name="rapports"),
    path("rapports/preparation/", JournalPreparationView.as_view(), name="preparation"),
    path("rapports/<uuid:pk>/", JournalRapportDetailView.as_view(), name="detail"),
    path("rapports/<uuid:pk>/draft/", JournalDraftView.as_view(), name="draft"),
    path("rapports/<uuid:pk>/soumettre/", JournalSoumettreView.as_view(), name="soumettre"),
    path("rapports/<uuid:pk>/valider/", JournalValiderView.as_view(), name="valider"),
    path("rapports/<uuid:pk>/approuver/", JournalApprouverView.as_view(), name="approuver"),
    path("rapports/<uuid:pk>/rejeter/", JournalRejeterView.as_view(), name="rejeter"),
    path("rapports/<uuid:pk>/alertes/", JournalAlertesView.as_view(), name="alertes"),
]
```

apps/chantier/migrations/0005_alertejournal_journalchantier_and_more.py

```
# Generated by Django 5.2.17 on 2026-10-07 22:16
import django.db.models.deletion
import uuid
from django.conf import settings
from django.db import migrations, models

class Migration(migrations.Migration):
    dependencies = [
        ("chantier", "0004_blocage"),
        ("projets", "0029_merge_20261007_1307"),
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]
    operations = [
        migrations.CreateModel(
            name="AlerteJournal",
            fields=[
                (
                    "id",
                    models.UUIDField(
                        default=uuid.uuid4, editable=False, primary_key=True, serialize=False
                    ),
                ),
                ("cree_le", models.DateTimeField(auto_now_add=True, verbose_name="créé le")),
                ("modifie_le", models.DateTimeField(auto_now=True, verbose_name="modifié le")),
                (
                    "supprime_le",
                    models.DateTimeField(
                        blank=True,
                        db_index=True,
                        help_text="Non nul = ligne supprimée logiquement. Jamais de suppression physique.",
                        null=True,
                        verbose_name="supprimé le",
                    ),
                ),
                ("cle", models.CharField(max_length=100)),
                ("type", models.CharField(max_length=30)),
                ("description", models.TextField()),
            ],
            options={
                "db_table": "alerte_journal",
            },
        ),
        migrations.CreateModel(
            name="JournalChantier",
            fields=[
                (
                    "id",
                    models.UUIDField(
                        default=uuid.uuid4, editable=False, primary_key=True, serialize=False
                    ),
                ),
                ("cree_le", models.DateTimeField(auto_now_add=True, verbose_name="créé le")),
                ("modifie_le", models.DateTimeField(auto_now=True, verbose_name="modifié le")),
                (
                    "supprime_le",
                    models.DateTimeField(
                        blank=True,
                        db_index=True,
                        help_text="Non nul = ligne supprimée logiquement. Jamais de suppression physique.",
                        null=True,
                        verbose_name="supprimé le",
                    ),
                ),
                ("saisie", models.JSONField(default=dict)),
                ("photographie", models.JSONField(default=dict)),
            ],
        ),
    ]
```

```

        ("valide_ct_le", models.DateTimeField(blank=True, null=True)),
        ("commentaire_ct", models.TextField(blank=True)),
        ("relevance_le", models.DateTimeField(blank=True, null=True)),
    ],
    options={
        "db_table": "journal_chantier",
    },
),
migrations.RemoveConstraint(
    model_name="rapportjournalier",
    name="uq_rapport_lot_date",
),
migrations.RemoveConstraint(
    model_name="rapportjournalier",
    name="uq_rapport_projet_date",
),
migrations.AddConstraint(
    model_name="rapportjournalier",
    constraint=models.UniqueConstraint(
        condition=models.Q(("lot__isnull", False), ("supprime_le__isnull", True)),
        fields=("lot", "date_rapport"),
        name="uq_rapport_lot_date",
    ),
),
migrations.AddConstraint(
    model_name="rapportjournalier",
    constraint=models.UniqueConstraint(
        condition=models.Q(("lot__isnull", True), ("supprime_le__isnull", True)),
        fields=("projet", "date_rapport"),
        name="uq_rapport_projet_date",
    ),
),
migrations.AddField(
    model_name="alertejournal",
    name="cree_par",
    field=models.ForeignKey(
        blank=True,
        help_text="Nul lorsque l'écriture vient du système.",
        null=True,
        on_delete=django.db.models.deletion.RESTRICT,
        related_name="+",
        to=settings.AUTH_USER_MODEL,
        verbose_name="créé par",
    ),
),
migrations.AddField(
    model_name="alertejournal",
    name="destinataires",
    field=models.ManyToManyField(
        related_name="alertes_journal", to=settings.AUTH_USER_MODEL
    ),
),
migrations.AddField(
    model_name="alertejournal",
    name="supprime_par",
    field=models.ForeignKey(
        blank=True,
        null=True,
        on_delete=django.db.models.deletion.RESTRICT,
        related_name="+",
        to=settings.AUTH_USER_MODEL,
        verbose_name="supprimé par",
    ),
),
migrations.AddField(
    model_name="journalchantier",
    name="cree_par",
    field=models.ForeignKey(

```

```

        blank=True,
        help_text="Nul lorsque l'écriture vient du système.",
        null=True,
        on_delete=django.db.models.deletion.RESTRICT,
        related_name="+",
        to=settings.AUTH_USER_MODEL,
        verbose_name="créé par",
    ),
),
migrations.AddField(
    model_name="journalchantier",
    name="rapport",
    field=models.OneToOneField(
        on_delete=django.db.models.deletion.RESTRICT,
        related_name="journal",
        to="chantier.rapportjournalier",
    ),
),
migrations.AddField(
    model_name="journalchantier",
    name="supprime_par",
    field=models.ForeignKey(
        blank=True,
        null=True,
        on_delete=django.db.models.deletion.RESTRICT,
        related_name="+",
        to=settings.AUTH_USER_MODEL,
        verbose_name="supprimé par",
    ),
),
migrations.AddField(
    model_name="journalchantier",
    name="valide_ct_par",
    field=models.ForeignKey(
        blank=True,
        null=True,
        on_delete=django.db.models.deletion.RESTRICT,
        related_name="journaux_valides_ct",
        to=settings.AUTH_USER_MODEL,
    ),
),
migrations.AddField(
    model_name="alertejournal",
    name="journal",
    field=models.ForeignKey(
        on_delete=django.db.models.deletion.RESTRICT,
        related_name="alertes",
        to="chantier.journalchantier",
    ),
),
migrations.AddConstraint(
    model_name="alertejournal",
    constraint=models.UniqueConstraint(
        fields=("journal", "cle"), name="uq_alerte_journal_cle"
    ),
),
]

```

apps/chantier/tests/test_journal_api.py

```
"""Parcours métier et refus d'accès du journal complet, dans un vrai tenant."""
import base64
import io
from datetime import timedelta
from uuid import uuid4
import pytest
from django_tenants.utils import schema_context
from PIL import Image
from rest_framework.test import APIClient
from apps.accounts.models import Role, Utilisateur
from apps.accounts.services.roles import initialiser_roles_par_defaut
from apps.chantier.models import AlerteJournal, JournalChantier, RapportJournalier
from apps.chantier.services.journal import aujourd'hui
from apps.projets.models import Activite, AffectationProjet, Lot, Projet
API = "/api/v1/chantier"
pytestmark = pytest.mark.django_db
@pytest.fixture
def terrain(db, schema_demo):
    initialiser_roles_par_defaut()
    personnes = {}
    for code in ("CC", "CT", "CP", "VI"):
        personnes[code] = Utilisateur.objects.create_user(
            email=f"journal-{code}-{uuid4().hex[:8]}@demo.ci",
            password="TestJournal!123",
            nom=code,
            prenom="Journal",
            role=Role.objects.get(code=code),
            role_global=code,
            statut="ACTIF",
        )
    projet = Projet.objects.create(
        reference=f"PRJ-J-{uuid4().hex[:6]}",
        nom="Journal chantier",
        ville="Abidjan",
        statut="EN_COURS",
        chef_projet=personnes["CP"],
        date_debut_prevue=aujourd'hui() - timedelta(days=10),
        date_fin_prevue=aujourd'hui() + timedelta(days=30),
    )
    for code, utilisateur in personnes.items():
        AffectationProjet.objects.create(projet=projet, utilisateur=utilisateur, role_projet=code)
    lot = Lot.objects.create(
        projet=projet, code="L-01", libelle="Gros œuvre", mode_execution="REGIE"
    )
    activite = Activite.objects.create(lot=lot, libelle="Béton", quantite_prevue=100, unite="M3")
    clients = {}
    for code, utilisateur in personnes.items():
        client = APIClient(HTTP_HOST="demo.localhost")
        client.force_authenticate(user=utilisateur)
        clients[code] = client
    return {**personnes, "projet": projet, "lot": lot, "activite": activite, "clients": clients}
def corps(terrain, **changements):
    data = {
        "projet_id": str(terrain["projet"].id),
        "date": aujourd'hui().isoformat(),
        "heure_debut": "07:30",
        "heure_fin": "17:00",
        "arret": None,
        "meteo": {
            "matin": "NUAGEUX",
            "apres_midi": "ORAGEUX",
            "conditions": "DIFFICILES",
            "temperature_min": 22,
            "temperature_max": 31,
            "humidite": 80,
            "vent": "Sud-ouest",
        }
    }
```

```

        "prevision": "Pluie",
    },
    "effectifs": [
        {
            "categorie": "Maçons",
            "prevus": 4,
            "resents": 4,
            "retards": 0,
            "heures": 8,
            "observation": "",
        }
    ],
    "lots_travaillés": [
        {"lot_id": str(terrain["lot"].id), "observation": "Coulage de la dalle"}
    ],
    "activites": [
        {
            "activite_id": str(terrain["activite"].id),
            "quantite_jour": 12.5,
            "localisation": "Bâtiment B, R+1",
            "observation": "Coulage conforme",
        }
    ],
    "materiaux": [{"designation": "Ciment", "quantite_consommee": 15, "unite": "sacs"}],
    "livraisons": [
        {
            "fournisseur": "Fournisseur CI",
            "designation": "Gravier",
            "quantite": "5 m3",
            "bon_livraison": "BL-01",
            "heure": "09:30",
            "conformite": "CONFORME",
            "observation": "",
        }
    ],
    "besoins": [
        {
            "nature": "URGENT",
            "designation": "Ciment",
            "quantite": "10 sacs",
            "observation": "Pour demain",
        }
    ],
    "equipements": [
        {
            "designation": "Bétonnière",
            "reference": "ENG-1",
            "propriete": "ENTREPRISE",
            "utilisation": "8 h",
            "opérateur": "Koné",
            "etat": "BON",
            "duree_arret": 0,
            "observation": "",
        }
    ],
    "incidents": [
        {
            "cle": "evt-1",
            "nature": "INTEMPERIE",
            "type": "MATERIEL",
            "heure_debut": "14:00",
            "heure_fin": "15:00",
            "gravite": "MINEUR",
            "decide_par": "CC",
            "description": "Pluie forte pendant une heure sur le chantier.",
            "action_entreprise": "Protection du matériel",
        }
    ],
],

```

```

        "blocage": {"niveau": "AUCUN", "nature": None, "description": "", "impact": ""},
        "previsions": [
            {
                "activite": "Coffrage",
                "equipe": "Coffreurs",
                "objectif": "Terminer la travée B",
                "prerequis": "Matériel disponible",
            }
        ],
        "note_cc": "Travaux réalisés conformément au planning.",
    }
    return {**data, **changements}

def creer(terrain, data=None):
    response = terrain["clients"]["CC"].post(
        f"{API}/rapports/", data or corps(terrain), format="json"
    )
    assert response.status_code == 201, response.data
    return response.data["id"]

def soumettre(terrain, identifiant):
    response = terrain["clients"]["CC"].post(
        f"{API}/rapports/{identifiant}/soumettre/", {}, format="json"
    )
    assert response.status_code == 200, response.data
    return response

def test_crud_et_reprise_du_formulaire(terrain):
    cc = terrain["clients"]["CC"]
    identifiant = creer(terrain)
    rep = cc.get(f"{API}/rapports/{identifiant}/")
    assert rep.status_code == 200
    assert rep.data["saisie"]["activites"][0]["localisation"] == "Bâtiment B, R+1"
    assert rep.data["travaux"][0]["activites"][0]["quantite_jour"] == 12.5
    assert rep.data["effectif_present"] == 4
    rep = cc.patch(
        f"{API}/rapports/{identifiant}/draft/",
        {"note_cc": "Note mise à jour", "meteo": {"vent": "Faible"}},
        format="json",
    )
    assert rep.status_code == 200, rep.data
    assert rep.data["saisie"]["meteo"]["matin"] == "NUAGEUX"
    assert rep.data["saisie"]["meteo"]["vent"] == "Faible"
    assert len(rep.data["saisie"]["activites"]) == 1
    rep = cc.get(
        f"{API}/rapports/preparation/",
        {"projet": str(terrain["projet"].id), "date": aujourd'hui().isoformat()},
    )
    assert rep.status_code == 200, rep.data
    assert rep.data["rapport"]["id"] == identifiant
    assert rep.data["lots"][0]["mode_execution"] == "REGIE_DIRECTE"
    assert rep.data["activites"][0]["activite_id"] == str(terrain["activite"].id)
    assert "EFFECTIFS" in rep.data["sections"]
    rep = cc.get(f"{API}/rapports/", {"auteur": "moi", "projet": str(terrain["projet"].id)})
    assert rep.status_code == 200
    assert rep.data["resultats"][0]["id"] == identifiant

def test_brouillon_incomplet_et_soumission_atomique(terrain):
    cc = terrain["clients"]["CC"]
    identifiant = creer(
        terrain, {"projet_id": str(terrain["projet"].id), "date": aujourd'hui().isoformat()}
    )
    rep = cc.post(
        f"{API}/rapports/{identifiant}/soumettre/",
        {"note_cc": "tentative incomplète"},
        format="json",
    )
    assert rep.status_code == 422, rep.data
    journal = JournalChantier.objects.get(rapport_id=identifiant)
    assert journal.saisie["note_cc"] == ""
    assert journal.rapport.statut == "BROUILLON"
    assert journal.rapport.soumis_le is None

```

```

def test_circuit_ct_cp_et_verrouillage(terrain):
    identifiant = creer(terrain)
    soumettre(terrain, identifiant)
    cc, ct, cp = (terrain["clients"][code] for code in ("CC", "CT", "CP"))
    rep = cc.patch(
        f"{API}/rapports/{identifiant}/", {"note_cc": "Modification interdite"}, format="json"
    )
    assert rep.status_code == 422
    rep = cp.post(f"{API}/rapports/{identifiant}/approuver/", {}, format="json")
    assert rep.status_code == 422
    rep = ct.post(
        f"{API}/rapports/{identifiant}/valider/", {"commentaire": "Conforme"}, format="json"
    )
    assert rep.status_code == 200, rep.data
    assert rep.data["statut"] == "VALIDE_CT"
    rep = ct.post(f"{API}/rapports/{identifiant}/approuver/", {}, format="json")
    assert rep.status_code == 403
    rep = cp.post(f"{API}/rapports/{identifiant}/approuver/", {}, format="json")
    assert rep.status_code == 200, rep.data
    assert rep.data["statut"] == "APPROUVE_CP"
    assert [e["etat"] for e in rep.data["circuit"]] == ["SIGNE", "SIGNE", "SIGNE"]
    rep = cp.post(
        f"{API}/rapports/{identifiant}/rejeter/",
        {"motif": "Un rapport approuvé est définitif."},
        format="json",
    )
    assert rep.status_code == 422
    terrain["lot"].refresh_from_db()
    assert terrain["lot"].premier_rapport_soumis

def test_rejet_et_resoumission(terrain):
    identifiant = creer(terrain)
    soumettre(terrain, identifiant)
    ct = terrain["clients"]["CT"]
    rep = ct.post(f"{API}/rapports/{identifiant}/rejeter/", {"motif": "court"}, format="json")
    assert rep.status_code == 400
    rep = ct.post(
        f"{API}/rapports/{identifiant}/rejeter/",
        {"motif": "Corriger la localisation des travaux réalisés."},
        format="json",
    )
    assert rep.status_code == 200, rep.data
    assert rep.data["statut"] == "REJETE"
    rep = terrain["clients"]["CC"].patch(
        f"{API}/rapports/{identifiant}/draft/",
        {"note_cc": "Localisation corrigée par le rédacteur."},
        format="json",
    )
    assert rep.status_code == 200
    soumettre(terrain, identifiant)
    journal = JournalChantier.objects.get(rapport_id=identifiant)
    assert journal.rapport.commentaire_validation == ""
    assert journal.valide_ct_le is None

def test_doublon_et_suppression_logique_brouillon(terrain):
    identifiant = creer(terrain)
    cc = terrain["clients"]["CC"]
    rep = cc.post(f"{API}/rapports/", corps(terrain), format="json")
    assert rep.status_code == 409
    rep = cc.delete(f"{API}/rapports/{identifiant}/")
    assert rep.status_code == 204, rep.data
    assert cc.get(f"{API}/rapports/{identifiant}/").status_code == 404
    assert RapportJournalier.tous_objets.get(pk=identifiant).supprime_le is not None
    nouveau = creer(terrain)
    assert nouveau != identifiant
    soumettre(terrain, nouveau)
    assert cc.delete(f"{API}/rapports/{nouveau}/").status_code == 422

@pytest.mark.parametrize(
    "changement",
    [

```

```

        {
            "effectifs": [
                {"categorie": "Maçons", "prevus": 2, "resents": 3, "retards": 0, "heures": 8}
            ]
        },
        {
            "effectifs": [
                {"categorie": "Maçons", "prevus": 2, "resents": 2, "retards": 3, "heures": 8}
            ]
        },
        {
            "effectifs": [
                {"categorie": "Maçons", "prevus": 2, "resents": 2, "retards": 0, "heures": 25}
            ]
        },
        {"meteo": {"matin": "INCONNU"}},
        {"meteo": {"temperature_min": 40, "temperature_max": 20}},
        {"lots_travaillés": [{"lot_id": str(uuid4()), "observation": "Extérieur"}]},
        {"activites": [{"activite_id": str(uuid4()), "quantite_jour": 2}]},
        {"statut": "APPROUVE_CP"},
    ],
)
def test_validation_structure_et_perimetre(terrain, changement):
    rep = terrain["clients"]["CC"].post(
        f"{API}/rapports/", corps(terrain, **changement), format="json"
    )
    assert rep.status_code == 400, rep.data
    assert not JournalChantier.objects.exists()
@pytest.mark.parametrize("jours", [-3, 1])
def test_fenetre_saisie(terrain, jours):
    rep = terrain["clients"]["CC"].post(
        f"{API}/rapports/",
        corps(terrain, date=(aujourd'hui() + timedelta(days=jours)).isoformat()),
        format="json",
    )
    assert rep.status_code == 422, rep.data
def test_acces_projet_auteur_et_tenant(terrain):
    identifiant = creer(terrain)
    ct = terrain["clients"]["CT"]
    assert ct.get(f"{API}/rapports/{identifiant}/").status_code == 404
    assert ct.get(f"{API}/rapports/").data["resultats"] == []
    autre = Projet.objects.create(
        reference=f"AUTRE-{uuid4().hex[:6]}", nom="Non affecté", ville="Abidjan", statut="EN_COURS"
    )
    cc = terrain["clients"]["CC"]
    rep = cc.post(
        f"{API}/rapports/",
        {"projet_id": str(autre.id), "date": aujourd'hui().isoformat()},
        format="json",
    )
    assert rep.status_code == 404
    soumettre(terrain, identifiant)
    assert ct.get(f"{API}/rapports/{identifiant}/").status_code == 200
    AffectationProjet.objects.filter(projet=terrain["projet"], utilisateur=terrain["CT"]).update(
        est_actif=False
    )
    assert ct.get(f"{API}/rapports/{identifiant}/").status_code == 404
    with schema_context("tenant_b"):
        assert not JournalChantier.objects.filter(rapport_id=identifiant).exists()
        anonyme = APIClient(HTTP_HOST="demo.localhost")
        assert anonyme.get(f"{API}/rapports/{identifiant}/").status_code == 401
def test_absence_motivée_et_journee_arret(terrain):
    identifiant = creer(
        terrain,
        corps(
            terrain,
            effectifs=[
                {

```

```

        "categorie": "Maçons",
        "prevus": 4,
        "presents": 3,
        "retards": 0,
        "heures": 8,
        "observation": "",
    },
],
),
)
rep = terrain["clients"]["CC"].post(
    f"{API}/rapports/{identifiant}/soumettre/", {}, format="json"
)
assert rep.status_code == 422
terrain["clients"]["CC"].delete(f"{API}/rapports/{identifiant}/")
data = {
    "projet_id": str(terrain["projet"].id),
    "date": aujourd'hui().isoformat(),
    "heure_debut": "07:30",
    "heure_fin": "17:00",
    "arret": {"motif": "INTERPERIES", "precision": "Pluie"},
    "meteo": {"matin": "PLUVIEUX", "apres_midi": "ORAGEUX", "conditions": "ARRET"},
}
identifiant = creer(terrain, data)
soumettre(terrain, identifiant)
terrain["activite"].refresh_from_db()
assert terrain["activite"].quantite_realisee == 0
def test_medias_embarques_et_protection(terrain):
    image = io.BytesIO()
    Image.new("RGB", (2, 2), "white").save(image, format="PNG")
    fichier = "data:image/png;base64," + base64.b64encode(image.getvalue()).decode()
    photo = {
        "cle": "photo-1",
        "fichier": fichier,
        "legende": "Dalle",
        "horodatage_utc": "2026-10-07T14:00:00Z",
        "latitude": 5.3,
        "longitude": -4.0,
    }
    pdf = b"%PDF-1.4\nDocument de test"
    piece = {
        "cle": "doc-1",
        "nom": "plan.pdf",
        "fichier": "data:application/pdf;base64," + base64.b64encode(pdf).decode(),
        "type_mime": "application/pdf",
        "taille": len(pdf),
    }
    identifiant = creer(terrain, corps(terrain, photos=[photo], pieces_jointes=[piece]))
    rep = terrain["clients"]["CC"].get(f"{API}/rapports/{identifiant}/")
    assert rep.data["photos"][0]["url"] == fichier
    assert rep.data["photos"][0]["gps_confirme"] is True
    assert rep.data["pieces_jointes"][0]["nom"] == "plan.pdf"
    assert terrain["clients"]["CT"].get(f"{API}/rapports/{identifiant}/").status_code == 404
    for invalide in (
        {"photo": "fichier": "https://example.com/a.png"},
        {"photo": "longitude": None},
        {"photo": "fichier": ""},
    ):
        rep = terrain["clients"]["CC"].patch(
            f"{API}/rapports/{identifiant}/draft/", {"photos": [invalide]}, format="json"
        )
        assert rep.status_code == 400, rep.data
    rep = terrain["clients"]["CC"].patch(
        f"{API}/rapports/{identifiant}/draft/",
        {"photos": [{"photo": "cle": str(i)} for i in range(6)]},
        format="json",
    )
    assert rep.status_code == 400

```

```

def test_snapshot_et_cumul_sans_double_comptage(terrain):
    identifiant = creer(
        terrain, corps(terrain, date=(aujourd'hui() - timedelta(days=1)).isoformat())
    )
    soumettre(terrain, identifiant)
    terrain["activite"].libelle = "Nouveau libellé"
    terrain["activite"].save()
    rep = terrain["clients"]["CC"].get(f"{API}/rapports/{identifiant}/")
    assert rep.data["travaux"][0]["activites"][0]["libelle"] == "Béton"
    rep = terrain["clients"]["CC"].get(
        f"{API}/rapports/preparation/",
        {"projet": str(terrain["projet"].id), "date": aujourd'hui().isoformat()},
    )
    assert rep.status_code == 200, rep.data
    assert rep.data["activites"][0]["cumul_veille"] == 12.5
def test_journal_rapports_manquants_et_filtres(terrain):
    identifiant = creer(terrain)
    rep = terrain["clients"]["CT"].get(
        f"{API}/journal/",
        {
            "date_debut": (aujourd'hui() - timedelta(days=2)).isoformat(),
            "date_fin": aujourd'hui().isoformat(),
            "projet": str(terrain["projet"].id),
        },
    )
    assert rep.status_code == 200, rep.data
    assert all(ligne["statut"] == "NON_SOUMIS" for ligne in rep.data["resultats"])
    assert all(ligne["note_chef_chantier"] is None for ligne in rep.data["resultats"])
    soumettre(terrain, identifiant)
    rep = terrain["clients"]["CT"].get(f"{API}/journal/")
    assert any(
        ligne["id"] == identifiant and ligne["statut"] == "SOUMIS"
        for ligne in rep.data["resultats"]
    )
    rep = terrain["clients"]["CC"].get(
        f"{API}/journal/", {"date_debut": "2026-10-08", "date_fin": "2026-10-07"}
    )
    assert rep.status_code == 400
def test_alerte_interne_idempotente(terrain):
    identifiant = creer(terrain)
    cc = terrain["clients"]["CC"]
    data = {
        "cle": "BLOCAGE",
        "type": "BLOCAGE_BLOQUANT",
        "description": "Travaux arrêtés pour raison de sécurité.",
    }
    rep = cc.post(f"{API}/rapports/{identifiant}/alertes/", data, format="json")
    assert rep.status_code == 200, rep.data
    assert rep.data["canal"] == "INTERNE"
    second = cc.post(f"{API}/rapports/{identifiant}/alertes/", data, format="json")
    assert second.data["envoyee_le"] == rep.data["envoyee_le"]
    assert AlerteJournal.objects.count() == 1
    assert set(AlerteJournal.objects.get().destinataires.values_list("id", flat=True)) == {
        terrain["CT"].id,
        terrain["CP"].id,
    }
    rep = terrain["clients"]["CT"].get(f"{API}/alertes/")
    assert rep.status_code == 200, rep.data
    assert rep.data["resultats"][0]["rapport_id"] == identifiant
    assert terrain["clients"]["CC"].get(f"{API}/alertes/").data["resultats"] == []
def test_production_et_masquage_financier(terrain):
    lot = Lot.objects.create(
        projet=terrain["projet"],
        code="ST-01",
        libelle="Tâcherons",
        mode_execution="SOUS_TRAITANCE_INFORMELLE",
    )
    activite = Activite.objects.create(lot=lot, libelle="Peinture", quantite_prevue=50, unite="M2")

```

```

production = [
    {
        "intervenant": "Koné",
        "activite_id": str(activite.id),
        "quantite_jour": 5,
        "prix_unitaire": None,
    }
]
identifiant = creer(terrain, corps(terrain, production=production))
cc = terrain["clients"]["CC"]
rep = cc.patch(
    f"{API}/rapports/{identifiant}/draft/",
    {"production": [{**production[0], "prix_unitaire": 5000}]},
    format="json",
)
assert rep.status_code == 400, rep.data
journal = JournalChantier.objects.get(rapport_id=identifiant)
# Tarif existant provenant d'une saisie financière autorisée.
journal.saisie["production"][0]["prix_unitaire"] = 5000
journal.save()
rep = cc.patch(
    f"{API}/rapports/{identifiant}/draft/", {"production": production}, format="json"
)
assert rep.status_code == 200, rep.data
assert rep.data["saisie"]["production"][0]["prix_unitaire"] is None
journal.refresh_from_db()
assert journal.saisie["production"][0]["prix_unitaire"] == 5000
soumettre(terrain, identifiant)
assert (
    terrain["clients"]["CT"]
    .get(f"{API}/rapports/{identifiant}/")
    .data["production"][0]["prix_unitaire"]
    is None
)
assert (
    terrain["clients"]["CP"]
    .get(f"{API}/rapports/{identifiant}/")
    .data["production"][0]["prix_unitaire"]
    == 5000
)
def test_audit_signatures_conserve_rejet_et_resoumission(terrain):
    from apps.audit.models import JournalAudit
    identifiant = creer(terrain)
    soumettre(terrain, identifiant)
    ct = terrain["clients"]["CT"]
    rep = ct.post(
        f"{API}/rapports/{identifiant}/rejeter/",
        {"motif": "Les observations doivent préciser la zone."},
        format="json",
    )
    assert rep.status_code == 200, rep.data
    soumettre(terrain, identifiant)
    evenements = list(
        JournalAudit.objects.filter(type_entite="JournalChantier", entite_id=identifiant)
        .order_by("horodatage")
        .values_list("valeur_apres", flat=True)
    )
    assert [e["evenement"] for e in evenements] == [
        "BROUILLON_CREE",
        "SOUMISSION_CC",
        "REJET",
        "SOUMISSION_CC",
    ]
    assert evenements[2]["commentaire"] == "Les observations doivent préciser la zone."
def test_autre_redacteur_et_validation_sans_permission(terrain):
    autre = Utilisateur.objects.create_user(
        email=f"journal-autre-{uuid4().hex[:8]}@demo.ci",
        nom="Autre",
    )

```

```

        role=Role.objects.get(code="CC"),
        role_global="CC",
        statut="ACTIF",
    )
    AffectationProjet.objects.create(projet=terrain["projet"], utilisateur=autre, role_projet="CC")
    client = APIClient(HTTP_HOST="demo.localhost")
    client.force_authenticate(user=autre)
    identifiant = creer(terrain)
    assert (
        client.patch(
            f"{API}/rapports/{identifiant}/draft/", {"note_cc": "Autre rédacteur"}, format="json"
        ).status_code
        == 404
    )
    soumettre(terrain, identifiant)
    assert (
        client.post(f"{API}/rapports/{identifiant}/soumettre/", {}, format="json").status_code
        == 403
    )
    assert (
        terrain["clients"]["VI"]
        .post(f"{API}/rapports/{identifiant}/valider/", {}, format="json")
        .status_code
        == 403
    )
def test_routes_historiques_ne_contournent_pas_le_circuit(terrain):
    identifiant = creer(terrain)
    soumettre(terrain, identifiant)
    rep = terrain["clients"]["CC"].patch(
        f"/api/v1/rapports/{identifiant}/", {"observations": "Contournement"}, format="json"
    )
    assert rep.status_code == 422
    rep = terrain["clients"]["CT"].post(
        f"/api/v1/rapports/{identifiant}/valider/", {}, format="json"
    )
    assert rep.status_code == 422
    assert RapportJournalier.objects.get(pk=identifiant).statut == "SOU MIS"

```

apps/chantier/tests/test_journal_validation.py

```
"""Validation pure des payloads : exécution rapide sans PostgreSQL."""
import pytest
from apps.chantier.serializers.journal import SaisieJournalSerializer
@pytest.mark.parametrize(
    "champ,valeur",
    [
        ("meteo", {"humidite": -1}),
        ("effectifs", [{"prevus": 3, "presents": 4}]),
        ("effectifs", [{"prevus": 4, "presents": 3, "retards": 4}]),
        ("production", [{"activite_id": "pas-un-uuid", "prix_unitaire": -1}]),
        ("photos", [{"cle": str(i)} for i in range(6)]),
        ("pieces_jointes", [{"cle": str(i)} for i in range(4)]),
        ("statut", "APPROUVE_CP"),
    ],
)
def test_entrees_invalides(champ, valeur):
    serializer = SaisieJournalSerializer(
        data={
            "projet_id": "c872c588-ec75-4776-9ebd-f0dcf44a84d7",
            "date": "2026-10-07",
            champ: valeur,
        }
    )
    assert not serializer.is_valid()
    assert champ in serializer.errors
def test_brouillon_vide_conserve_nombres_non_reseignes():
    serializer = SaisieJournalSerializer(
        data={
            "projet_id": "c872c588-ec75-4776-9ebd-f0dcf44a84d7",
            "date": "2026-10-07",
            "effectifs": [{"categorie": "Maçons"}],
        }
    )
    assert serializer.is_valid(), serializer.errors
    assert serializer.data["effectifs"][0]["presents"] is None
    assert serializer.data["photos"] == []
```